
Mypy Documentation

Release 2.3.0

Jukka

Jul 13, 2026

FIRST STEPS

1	Contents	3
1.1	Getting started	3
1.2	Type hints cheat sheet	7
1.3	Using mypy with an existing codebase	14
1.4	Built-in types	18
1.5	Type inference and type annotations	19
1.6	Kinds of types	23
1.7	Class basics	33
1.8	Annotation issues at runtime	39
1.9	Protocols and structural subtyping	44
1.10	Dynamically typed code	53
1.11	Type narrowing	55
1.12	Duck type compatibility	63
1.13	Stub files	64
1.14	Generics	66
1.15	More types	88
1.16	Literal types and Enums	103
1.17	TypedDict	111
1.18	Final names, methods and classes	117
1.19	Metaclasses	121
1.20	Running mypy and managing imports	123
1.21	The mypy command line	130
1.22	The mypy configuration file	147
1.23	Inline configuration	169
1.24	Mypy daemon (mypy server)	169
1.25	Using installed packages	174
1.26	Extending and integrating mypy	177
1.27	Automatic stub generation (stubgen)	180
1.28	Automatic stub testing (stubtest)	183
1.29	Common issues and solutions	186
1.30	Supported Python features	198
1.31	Error codes	198
1.32	Error codes enabled by default	200
1.33	Error codes for optional checks	219
1.34	Additional features	230
1.35	Frequently Asked Questions	236
1.36	Mypy Release Notes	239
2	Indices and tables	357

Mypy is a static type checker for Python.

Type checkers help ensure that you're using variables and functions in your code correctly. With mypy, add type hints ([PEP 484](#)) to your Python programs, and mypy will warn you when you use those types incorrectly.

Python is a dynamic language, so usually you'll only see errors in your code when you attempt to run it. Mypy is a *static* checker, so it finds bugs in your programs without even running them!

Here is a small example to whet your appetite:

```
number = input("What is your favourite number?")
print("It is", number + 1) # error: Unsupported operand types for + ("str" and "int")
```

Adding type hints for mypy does not interfere with the way your program would otherwise run. Think of type hints as similar to comments! You can always use the Python interpreter to run your code, even if mypy reports errors.

Mypy is designed with gradual typing in mind. This means you can add type hints to your code base slowly and that you can always fall back to dynamic typing when static typing is not convenient.

Mypy has a powerful and easy-to-use type system, supporting features such as type inference, generics, callable types, tuple types, union types, structural subtyping and more. Using mypy will make your programs easier to understand, debug, and maintain.

Note

Although mypy is production ready, there may be occasional changes that break backward compatibility. The mypy development team tries to minimize the impact of changes to user code. In case of a major breaking change, mypy's major version will be bumped.

CONTENTS

1.1 Getting started

This chapter introduces some core concepts of mypy, including function annotations, the `typing` module, stub files, and more.

If you're looking for a quick intro, see the [mypy cheatsheet](#).

If you're unfamiliar with the concepts of static and dynamic type checking, be sure to read this chapter carefully, as the rest of the documentation may not make much sense otherwise.

1.1.1 Installing and running mypy

Mypy requires Python 3.10 or later to run. You can install mypy using pip:

```
$ python3 -m pip install mypy
```

Once mypy is installed, run it by using the mypy tool:

```
$ mypy program.py
```

This command makes mypy *type check* your `program.py` file and print out any errors it finds. Mypy will type check your code *statically*: this means that it will check for errors without ever running your code, just like a linter.

This also means that you are always free to ignore the errors mypy reports, if you so wish. You can always use the Python interpreter to run your code, even if mypy reports errors.

However, if you try directly running mypy on your existing Python code, it will most likely report little to no errors. This is a feature! It makes it easy to adopt mypy incrementally.

In order to get useful diagnostics from mypy, you must add *type annotations* to your code. See the section below for details.

1.1.2 Dynamic vs static typing

A function without type annotations is considered to be *dynamically typed* by mypy:

```
def greeting(name):  
    return 'Hello ' + name
```

By default, mypy will **not** type check dynamically typed functions. This means that with a few exceptions, mypy will not report any errors with regular unannotated Python.

This is the case even if you misuse the function!

```
def greeting(name):
    return 'Hello ' + name

# These calls will fail when the program runs, but mypy does not report an error
# because "greeting" does not have type annotations.
greeting(123)
greeting(b"Alice")
```

We can get mypy to detect these kinds of bugs by adding *type annotations* (also known as *type hints*). For example, you can tell mypy that `greeting` both accepts and returns a string like so:

```
# The "name: str" annotation says that the "name" argument should be a string
# The "-> str" annotation says that "greeting" will return a string
def greeting(name: str) -> str:
    return 'Hello ' + name
```

This function is now *statically typed*: mypy will use the provided type hints to detect incorrect use of the `greeting` function and incorrect use of variables within the `greeting` function. For example:

```
def greeting(name: str) -> str:
    return 'Hello ' + name

greeting(3)           # Argument 1 to "greeting" has incompatible type "int"; expected "str"
# ~~~~~
greeting(b'Alice')    # Argument 1 to "greeting" has incompatible type "bytes"; expected
# ~~~~~
greeting("World!")    # No error

def bad_greeting(name: str) -> str:
    return 'Hello ' * name # Unsupported operand types for * ("str" and "str")
```

Being able to pick whether you want a function to be dynamically or statically typed can be very helpful. For example, if you are migrating an existing Python codebase to use static types, it's usually easier to migrate by incrementally adding type hints to your code rather than adding them all at once. Similarly, when you are prototyping a new feature, it may be convenient to initially implement the code using dynamic typing and only add type hints later once the code is more stable.

Once you are finished migrating or prototyping your code, you can make mypy warn you if you add a dynamic function by mistake by using the `--disallow-untyped-defs` flag. You can also get mypy to provide some limited checking of dynamically typed functions by using the `--check-untyped-defs` flag. See *The mypy command line* for more information on configuring mypy.

1.1.3 Strict mode and configuration

Mypy has a *strict mode* that enables a number of additional checks, like `--disallow-untyped-defs`.

If you run mypy with the `--strict` flag, you will basically never get a type related error at runtime without a corresponding mypy error, unless you explicitly circumvent mypy somehow.

However, this flag will probably be too aggressive if you are trying to add static types to a large, existing codebase. See *Using mypy with an existing codebase* for suggestions on how to handle that case.

Mypy is very configurable, so you can start with using `--strict` and toggle off individual checks. For instance, if you use many third party libraries that do not have types, `--ignore-missing-imports` may be useful. See *Introduce stricter options* for how to build up to `--strict`.

See *The mypy command line* and *The mypy configuration file* for a complete reference on configuration options.

1.1.4 More complex types

So far, we’ve added type hints that use only basic concrete types like `str` and `float`. What if we want to express more complex types, such as “a list of strings” or “an iterable of ints”?

For example, to indicate that some function can accept a list of strings, use the `list[str]` type (Python 3.9 and later):

```
def greet_all(names: list[str]) -> None:
    for name in names:
        print('Hello ' + name)

names = ["Alice", "Bob", "Charlie"]
ages = [10, 20, 30]

greet_all(names)    # Ok!
greet_all(ages)     # Error due to incompatible types
```

The `list` type is an example of something called a *generic type*: it can accept one or more *type parameters*. In this case, we *parameterized* `list` by writing `list[str]`. This lets mypy know that `greet_all` accepts specifically lists containing strings, and not lists containing ints or any other type.

In the above examples, the type signature is perhaps a little too rigid. After all, there’s no reason why this function must accept *specifically* a list – it would run just fine if you were to pass in a tuple, a set, or any other custom iterable.

You can express this idea using `collections.abc.Iterable`:

```
from collections.abc import Iterable # or "from typing import Iterable"

def greet_all(names: Iterable[str]) -> None:
    for name in names:
        print('Hello ' + name)
```

This behavior is actually a fundamental aspect of the PEP 484 type system: when we annotate some variable with a type `T`, we are actually telling mypy that variable can be assigned an instance of `T`, or an instance of a *subtype* of `T`. That is, `list[str]` is a subtype of `Iterable[str]`.

This also applies to inheritance, so if you have a class `Child` that inherits from `Parent`, then a value of type `Child` can be assigned to a variable of type `Parent`. For example, a `RuntimeError` instance can be passed to a function that is annotated as taking an `Exception`.

As another example, suppose you want to write a function that can accept *either* ints or strings, but no other types. You can express this using a union type. For example, `int` is a subtype of `int | str`:

```
def normalize_id(user_id: int | str) -> str:
    if isinstance(user_id, int):
        return f'user-{100_000 + user_id}'
    else:
        return user_id
```

The `typing` module contains many other useful types.

For a quick overview, look through the [mypy cheatsheet](#).

For a detailed overview (including information on how to make your own generic types or your own type aliases), look through the [type system reference](#).

Note

When adding types, the convention is to import types using the form `from typing import <name>` (as opposed to doing just `import typing` or `import typing as t` or `from typing import *`).

For brevity, we often omit imports from `typing` or `collections.abc` in code examples, but mypy will give an error if you use types such as `Iterable` without first importing them.

Note

In some examples we use capitalized variants of types, such as `List`, and sometimes we use plain `list`. They are equivalent, but the prior variant is needed if you are using Python 3.8 or earlier.

1.1.5 Local type inference

Once you have added type hints to a function (i.e. made it statically typed), mypy will automatically type check that function's body. While doing so, mypy will try and *infer* as many details as possible.

We saw an example of this in the `normalize_id` function above – mypy understands basic `isinstance` checks and so can infer that the `user_id` variable was of type `int` in the if-branch and of type `str` in the else-branch.

As another example, consider the following function. Mypy can type check this function without a problem: it will use the available context and deduce that `output` must be of type `list[float]` and that `num` must be of type `float`:

```
def nums_below(numbers: Iterable[float], limit: float) -> list[float]:
    output = []
    for num in numbers:
        if num < limit:
            output.append(num)
    return output
```

For more details, see *Type inference and type annotations*.

1.1.6 Types from libraries

Mypy can also understand how to work with types from libraries that you use.

For instance, mypy comes out of the box with an intimate knowledge of the Python standard library. For example, here is a function which uses the `Path` object from the `pathlib` standard library module:

```
from pathlib import Path

def load_template(template_path: Path, name: str) -> str:
    # Mypy knows that `template_path` has a `read_text` method that returns a str
    template = template_path.read_text()
    # ...so it understands this line type checks
    return template.replace('USERNAME', name)
```

If a third party library you use *declares support for type checking*, mypy will type check your use of that library based on the type hints it contains.

However, if the third party library does not have type hints, mypy will complain about missing type information.

```

prog.py:1: error: Library stubs not installed for "yaml"
prog.py:1: note: Hint: "python3 -m pip install types-PyYAML"
prog.py:2: error: Library stubs not installed for "requests"
prog.py:2: note: Hint: "python3 -m pip install types-requests"
...

```

In this case, you can provide mypy a different source of type information, by installing a *stub* package. A stub package is a package that contains type hints for another library, but no actual code.

```
$ python3 -m pip install types-PyYAML types-requests
```

Stubs packages for a distribution are often named `types-<distribution>`. Note that a distribution name may be different from the name of the package that you import. For example, `types-PyYAML` contains stubs for the `yaml` package.

For more discussion on strategies for handling errors about libraries without type information, refer to [Missing imports](#).

For more information about stubs, see [Stub files](#).

1.1.7 Next steps

If you are in a hurry and don't want to read lots of documentation before getting started, here are some pointers to quick learning resources:

- Read the [mypy cheatsheet](#).
- Read [Using mypy with an existing codebase](#) if you have a significant existing codebase without many type annotations.
- Read the [blog post](#) about the Zulip project's experiences with adopting mypy.
- If you prefer watching talks instead of reading, here are some ideas:
 - Carl Meyer: [Type Checked Python in the Real World](#) (PyCon 2018)
 - Greg Price: [Clearer Code at Scale: Static Types at Zulip and Dropbox](#) (PyCon 2018)
- Look at [solutions to common issues](#) with mypy if you encounter problems.
- You can ask questions about mypy in the [mypy issue tracker](#) and typing [Gitter chat](#).
- For general questions about Python typing, try posting at [typing discussions](#).

You can also continue reading this document and skip sections that aren't relevant for you. You don't need to read sections in order.

1.2 Type hints cheat sheet

This document is a quick cheat sheet showing how to use type annotations for various common types in Python.

1.2.1 Variables

Technically many of the type annotations shown below are redundant, since mypy can usually infer the type of a variable from its value. See [Type inference and type annotations](#) for more details.

```

# This is how you declare the type of a variable
age: int = 1

# You don't need to initialize a variable to annotate it

```

(continues on next page)

(continued from previous page)

```

a: int # Ok (no value at runtime until assigned)

# Doing so can be useful in conditional branches
child: bool
if age < 18:
    child = True
else:
    child = False

```

1.2.2 Useful built-in types

```

# For most types, just use the name of the type in the annotation
# Note that mypy can usually infer the type of a variable from its value,
# so technically these annotations are redundant
x: int = 1
x: float = 1.0
x: bool = True
x: str = "test"
x: bytes = b"test"

# For collections, the type of the collection item is in brackets
x: list[int] = [1]
x: set[int] = {6, 7}

# For mappings, we need the types of both keys and values
x: dict[str, float] = {"field": 2.0}

# For tuples of fixed size, we specify the types of all the elements
x: tuple[int, str, float] = (3, "yes", 7.5)

# For tuples of variable size, we use one type and ellipsis
x: tuple[int, ...] = (1, 2, 3)

# On Python 3.8 and earlier, the name of the collection type is
# capitalized, and the type is imported from the 'typing' module
from typing import List, Set, Dict, Tuple
x: List[int] = [1]
x: Set[int] = {6, 7}
x: Dict[str, float] = {"field": 2.0}
x: Tuple[int, str, float] = (3, "yes", 7.5)
x: Tuple[int, ...] = (1, 2, 3)

from typing import Union, Optional

# Use the | operator when something could be one of a few types
x: list[int | str] = [3, 5, "test", "fun"]
# Union is equivalent
x: list[Union[int, str]] = [3, 5, "test", "fun"]

# Use X | None for a value that could be None; Optional[X] is the same as X | None
x: str | None = "something" if some_condition() else None

```

(continues on next page)

(continued from previous page)

```

if x is not None:
    # Mypy understands x won't be None here because of the if-statement
    print(x.upper())
# If you know a value can never be None due to some logic that mypy doesn't
# understand, use an assert
assert x is not None
print(x.upper())

```

1.2.3 Functions

```

from collections.abc import Iterator, Callable
from typing import Union, Optional

# This is how you annotate a function definition
def stringify(num: int) -> str:
    return str(num)

# And here's how you specify multiple arguments
def plus(num1: int, num2: int) -> int:
    return num1 + num2

# If a function does not return a value, use None as the return type
# Default value for an argument goes after the type annotation
def show(value: str, excitement: int = 10) -> None:
    print(value + "!" * excitement)

# Note that arguments without a type are dynamically typed (treated as Any)
# and that functions without any annotations are not checked
def untyped(x):
    x.anything() + 1 + "string" # no errors

# This is how you annotate a callable (function) value
x: Callable[[int, float], float] = f
def register(callback: Callable[[str], int]) -> None: ...

# A generator function that yields ints is secretly just a function that
# returns an iterator of ints, so that's how we annotate it
def gen(n: int) -> Iterator[int]:
    i = 0
    while i < n:
        yield i
        i += 1

# You can of course split a function annotation over multiple lines
def send_email(
    address: str | list[str],
    sender: str,
    cc: list[str] | None,
    bcc: list[str] | None,
    subject: str = '',
    body: list[str] | None = None,

```

(continues on next page)

(continued from previous page)

```

) -> bool:
    ...

# Mypy understands positional-only and keyword-only arguments
# Positional-only arguments can also be marked by using a name starting with
# two underscores
def quux(x: int, /, *, y: int) -> None:
    pass

quux(3, y=5) # Ok
quux(3, 5) # error: Too many positional arguments for "quux"
quux(x=3, y=5) # error: Unexpected keyword argument "x" for "quux"

# This says each positional arg and each keyword arg is a "str"
def call(self, *args: str, **kwargs: str) -> str:
    reveal_type(args) # Revealed type is "tuple[str, ...]"
    reveal_type(kwargs) # Revealed type is "dict[str, str]"
    request = make_request(*args, **kwargs)
    return self.do_api_query(request)

```

1.2.4 Classes

```

from typing import ClassVar

class BankAccount:
    # The "__init__" method doesn't return anything, so it gets return
    # type "None" just like any other method that doesn't return anything
    def __init__(self, account_name: str, initial_balance: int = 0) -> None:
        # mypy will infer the correct types for these instance variables
        # based on the types of the parameters.
        self.account_name = account_name
        self.balance = initial_balance

    # For instance methods, omit type for "self"
    def deposit(self, amount: int) -> None:
        self.balance += amount

    def withdraw(self, amount: int) -> None:
        self.balance -= amount

# User-defined classes are valid as types in annotations
account: BankAccount = BankAccount("Alice", 400)
def transfer(src: BankAccount, dst: BankAccount, amount: int) -> None:
    src.withdraw(amount)
    dst.deposit(amount)

# Functions that accept BankAccount also accept any subclass of BankAccount!
class AuditedBankAccount(BankAccount):
    # You can optionally declare instance variables in the class body
    audit_log: list[str]

```

(continues on next page)

(continued from previous page)

```

def __init__(self, account_name: str, initial_balance: int = 0) -> None:
    super().__init__(account_name, initial_balance)
    self.audit_log: list[str] = []

def deposit(self, amount: int) -> None:
    self.audit_log.append(f"Deposited {amount}")
    self.balance += amount

def withdraw(self, amount: int) -> None:
    self.audit_log.append(f"Withdrew {amount}")
    self.balance -= amount

audited = AuditedBankAccount("Bob", 300)
transfer(audited, account, 100) # type checks!

# You can use the ClassVar annotation to declare a class variable
class Car:
    seats: ClassVar[int] = 4
    passengers: ClassVar[list[str]]

# If you want dynamic attributes on your class, have it
# override "__setattr__" or "__getattr__"
class A:
    # This will allow assignment to any A.x, if x is the same type as "value"
    # (use "value: Any" to allow arbitrary types)
    def __setattr__(self, name: str, value: int) -> None: ...

    # This will allow access to any A.x, if x is compatible with the return type
    def __getattr__(self, name: str) -> int: ...

a = A()
a.foo = 42 # Works
a.bar = 'Ex-parrot' # Fails type checking

```

1.2.5 When you're puzzled or when things are complicated

```

from typing import Union, Any, Optional, TYPE_CHECKING, cast

# To find out what type mypy infers for an expression anywhere in
# your program, wrap it in reveal_type(). Mypy will print an error
# message with the type; remove it again before running the code.
reveal_type(1) # Revealed type is "builtins.int"

# If you initialize a variable with an empty container or "None"
# you may have to help mypy a bit by providing an explicit type annotation
x: list[str] = []
x: str | None = None

# Use Any if you don't know the type of something or it's too
# dynamic to write a type for
x: Any = mystery_function()

```

(continues on next page)

(continued from previous page)

```

# Mypy will let you do anything with x!
x.whatever() * x["you"] + x("want") - any(x) and all(x) is super # no errors

# Use a "type: ignore" comment to suppress errors on a given line,
# when your code confuses mypy or runs into an outright bug in mypy.
# Good practice is to add a comment explaining the issue.
x = confusing_function() # type: ignore # confusing_function won't return None here_
↳ because ...

# "cast" is a helper function that lets you override the inferred
# type of an expression. It's only for mypy -- there's no runtime check.
a = [4]
b = cast(list[int], a) # Passes fine
c = cast(list[str], a) # Passes fine despite being a lie (no runtime check)
reveal_type(c) # Revealed type is "builtins.list[builtins.str]"
print(c) # Still prints [4] ... the object is not changed or casted at runtime

# Use "TYPE_CHECKING" if you want to have code that mypy can see but will not
# be executed at runtime (or to have code that mypy can't see)
if TYPE_CHECKING:
    import json
else:
    import orjson as json # mypy is unaware of this

```

In some cases type annotations can cause issues at runtime, see *Annotation issues at runtime* for dealing with this.

See *Silencing type errors* for details on how to silence errors.

1.2.6 Standard “duck types”

In typical Python code, many functions that can take a list or a dict as an argument only need their argument to be somehow “list-like” or “dict-like”. A specific meaning of “list-like” or “dict-like” (or something-else-like) is called a “duck type”, and several duck types that are common in idiomatic Python are standardized.

```

from collections.abc import Mapping, MutableMapping, Sequence, Iterable
# or 'from typing import ...' (required in Python 3.8)

# Use Iterable for generic iterables (anything usable in "for"),
# and Sequence where a sequence (supporting "len" and "__getitem__") is
# required
def f(ints: Iterable[int]) -> list[str]:
    return [str(x) for x in ints]

f(range(1, 3))

# Mapping describes a dict-like object (with "__getitem__") that we won't
# mutate, and MutableMapping one (with "__setitem__") that we might
def f(my_mapping: Mapping[int, str]) -> list[int]:
    my_mapping[5] = 'maybe' # mypy will complain about this line...
    return list(my_mapping.keys())

f({3: 'yes', 4: 'no'})

```

(continues on next page)

(continued from previous page)

```
def f(my_mapping: MutableMapping[int, str]) -> set[str]:
    my_mapping[5] = 'maybe' # ...but mypy is OK with this.
    return set(my_mapping.values())

f({3: 'yes', 4: 'no'})

import sys
from typing import IO

# Use IO[str] or IO[bytes] for functions that should accept or return
# objects that come from an open() call (note that IO does not
# distinguish between reading, writing or other modes)
def get_sys_IO(mode: str = 'w') -> IO[str]:
    if mode == 'w':
        return sys.stdout
    elif mode == 'r':
        return sys.stdin
    else:
        return sys.stdout
```

You can even make your own duck types using *Protocols and structural subtyping*.

1.2.7 Forward references

```
# You may want to reference a class before it is defined.
# This is known as a "forward reference".
def f(foo: A) -> int: # This will fail at runtime with 'A' is not defined
    ...

# However, if you add the following special import:
from __future__ import annotations
# It will work at runtime and type checking will succeed as long as there
# is a class of that name later on in the file
def f(foo: A) -> int: # Ok
    ...

# Another option is to just put the type in quotes
def f(foo: 'A') -> int: # Also ok
    ...

class A:
    # This can also come up if you need to reference a class in a type
    # annotation inside the definition of that class
    @classmethod
    def create(cls) -> A:
        ...
```

See *Class name forward references* for more details.

1.2.8 Decorators

Decorator functions can be expressed via generics. See *Declaring decorators* for more details. Example using Python 3.12 syntax:

```
from collections.abc import Callable
from typing import Any

def bare_decorator[F: Callable[..., Any]](func: F) -> F:
    ...

def decorator_args[F: Callable[..., Any]](url: str) -> Callable[[F], F]:
    ...
```

The same example using pre-3.12 syntax:

```
from collections.abc import Callable
from typing import Any, TypeVar

F = TypeVar('F', bound=Callable[..., Any])

def bare_decorator(func: F) -> F:
    ...

def decorator_args(url: str) -> Callable[[F], F]:
    ...
```

1.2.9 Coroutines and asyncio

See *Typing async/await* for the full detail on typing coroutines and asynchronous code.

```
import asyncio

# A coroutine is typed like a normal function
async def countdown(tag: str, count: int) -> str:
    while count > 0:
        print(f'T-minus {count} ({tag})')
        await asyncio.sleep(0.1)
        count -= 1
    return "Blastoff!"
```

1.3 Using mypy with an existing codebase

This section explains how to get started using mypy with an existing, significant codebase that has little or no type annotations. If you are a beginner, you can skip this section.

1.3.1 Start small

If your codebase is large, pick a subset of your codebase (say, 5,000 to 50,000 lines) and get mypy to run successfully only on this subset at first, *before adding annotations*. This should be doable in a day or two. The sooner you get some form of mypy passing on your codebase, the sooner you benefit.

You'll likely need to fix some mypy errors, either by inserting annotations requested by mypy or by adding `# type: ignore` comments to silence errors you don't want to fix now.

We'll mention some tips for getting mypy passing on your codebase in various sections below.

1.3.2 Run mypy consistently and prevent regressions

Make sure all developers on your codebase run mypy the same way. One way to ensure this is adding a small script with your mypy invocation to your codebase, or adding your mypy invocation to existing tools you use to run tests, like `tox`.

- Make sure everyone runs mypy with the same options. Checking a mypy [configuration file](#) into your codebase is the easiest way to do this.
- Make sure everyone type checks the same set of files. See [Specifying code to be checked](#) for details.
- Make sure everyone runs mypy with the same version of mypy, for instance by pinning mypy with the rest of your dev requirements.

In particular, you'll want to make sure to run mypy as part of your Continuous Integration (CI) system as soon as possible. This will prevent new type errors from being introduced into your codebase.

A simple CI script could look something like this:

```
python3 -m pip install mypy==1.8
# Run your standardised mypy invocation, e.g.
mypy my_project
# This could also look like `scripts/run_mypy.sh`, `tox run -e mypy`, `make mypy`, etc
```

1.3.3 Ignoring errors from certain modules

By default mypy will follow imports in your code and try to check everything. This means even if you only pass in a few files to mypy, it may still process a large number of imported files. This could potentially result in lots of errors you don't want to deal with at the moment.

One way to deal with this is to ignore errors in modules you aren't yet ready to type check. The [ignore_errors](#) option is useful for this, for instance, if you aren't yet ready to deal with errors from `package_to_fix_later`:

```
[mypy-package_to_fix_later.*]
ignore_errors = True
```

You could even invert this, by setting `ignore_errors = True` in your global config section and only enabling error reporting with `ignore_errors = False` for the set of modules you are ready to type check.

The per-module configuration that mypy's configuration file allows can be extremely useful. Many configuration options can be enabled or disabled only for specific modules. In particular, you can also enable or disable various error codes on a per-module basis, see [Error codes](#).

1.3.4 Fixing errors related to imports

A common class of error you will encounter is errors from mypy about modules that it can't find, that don't have types, or don't have stub files:

```
core/config.py:7: error: Cannot find implementation or library stub for module named
→ 'froblicate'
core/model.py:9: error: Cannot find implementation or library stub for module named 'acme
→ '
...
```

Sometimes these can be fixed by installing the relevant packages or stub libraries in the environment you're running mypy in.

See [Missing imports](#) for a complete reference on these errors and the ways in which you can fix them.

You'll likely find that you want to suppress all errors from importing a given module that doesn't have types. If you only import that module in one or two places, you can use `# type: ignore` comments. For example, here we ignore an error about a third-party module `frobnicate` that doesn't have stubs using `# type: ignore`:

```
import frobnicate # type: ignore
...
frobnicate.initialize() # OK (but not checked)
```

But if you import the module in many places, this becomes unwieldy. In this case, we recommend using a [configuration file](#). For example, to disable errors about importing `frobnicate` and `acme` everywhere in your codebase, use a config like this:

```
[mypy-frobnicate.*]
ignore_missing_imports = True

[mypy-acme.*]
ignore_missing_imports = True
```

If you get a large number of errors, you may want to ignore all errors about missing imports, for instance by setting `--disable-error-code=import-untyped`. or setting `ignore_missing_imports` to true globally. This can hide errors later on, so we recommend avoiding this if possible.

Finally, mypy allows fine-grained control over specific import following behaviour. It's very easy to silently shoot yourself in the foot when playing around with these, so this should be a last resort. For more details, look [here](#).

1.3.5 Prioritise annotating widely imported modules

Most projects have some widely imported modules, such as utilities or model classes. It's a good idea to annotate these pretty early on, since this allows code using these modules to be type checked more effectively.

Mypy is designed to support gradual typing, i.e. letting you add annotations at your own pace, so it's okay to leave some of these modules unannotated. The more you annotate, the more useful mypy will be, but even a little annotation coverage is useful.

1.3.6 Write annotations as you go

Consider adding something like these in your code style conventions:

1. Developers should add annotations for any new code.
2. It's also encouraged to write annotations when you modify existing code.

This way you'll gradually increase annotation coverage in your codebase without much effort.

1.3.7 Automate annotation of legacy code

There are tools for automatically adding draft annotations based on simple static analysis or on type profiles collected at runtime. Tools include [MonkeyType](#), [autotyping](#) and [PyAnnotate](#).

A simple approach is to collect types from test runs. This may work well if your test coverage is good (and if your tests aren't very slow).

Another approach is to enable type collection for a small, random fraction of production network requests. This clearly requires more care, as type collection could impact the reliability or the performance of your service.

1.3.8 Introduce stricter options

Mypy is very configurable. Once you get started with static typing, you may want to explore the various strictness options mypy provides to catch more bugs. For example, you can ask mypy to require annotations for all functions in certain modules to avoid accidentally introducing code that won't be type checked using [disallow_untyped_defs](#). Refer to [The mypy configuration file](#) for the details.

An excellent goal to aim for is to have your codebase pass when run against `mypy --strict`. This basically ensures that you will never have a type related error without an explicit circumvention somewhere (such as a `# type: ignore` comment).

The following config is equivalent to `--strict` (as of mypy 1.0):

```
# Start off with these
warn_unused_configs = True
warn_redundant_casts = True
warn_unused_ignores = True

# Getting this passing should be easy
strict_equality = True

# Strongly recommend enabling this one as soon as you can
check_untyped_defs = True

# These shouldn't be too much additional work, but may be tricky to
# get passing if you use a lot of untyped libraries
disallow_subclassing_any = True
disallow_untyped_decorators = True
disallow_any_generics = True

# These next few are various gradations of forcing use of type annotations
disallow_untyped_calls = True
disallow_incomplete_defs = True
disallow_untyped_defs = True

# This one isn't too hard to get passing, but return on investment is lower
no_implicit_reexport = True

# This one can be tricky to get passing if you use a lot of untyped libraries
warn_return_any = True

# This one is a catch-all flag for the rest of strict checks that are technically
# correct but may not be practical
extra_checks = True
```

Note that you can also start with `--strict` and subtract, for instance:

```
strict = True
warn_return_any = False
```

Remember that many of these options can be enabled on a per-module basis. For instance, you may want to enable `disallow_untyped_defs` for modules which you've completed annotations for, in order to prevent new code from being added without annotations.

And if you want, it doesn't stop at `--strict`. Mypy has additional checks that are not part of `--strict` that can be useful. See the complete [The mypy command line](#) reference and [Error codes for optional checks](#).

1.3.9 Speed up mypy runs

You can use *mypy daemon* to get much faster incremental mypy runs. The larger your project is, the more useful this will be. If your project has at least 100,000 lines of code or so, you may also want to set up *remote caching* for further speedups.

1.4 Built-in types

This chapter introduces some commonly used built-in types. We will cover many other kinds of types later.

1.4.1 Simple types

Here are examples of some common built-in types:

Type	Description
<code>int</code>	integer
<code>float</code>	floating point number
<code>bool</code>	boolean value (subclass of <code>int</code>)
<code>str</code>	text, sequence of unicode codepoints
<code>bytes</code>	8-bit string, sequence of byte values
<code>object</code>	an arbitrary object (<code>object</code> is the common base class)

All built-in classes can be used as types.

1.4.2 Any type

If you can't find a good type for some value, you can always fall back to `Any`:

Type	Description
<code>Any</code>	dynamically typed value with an arbitrary type

The type `Any` is defined in the `typing` module. See *Dynamically typed code* for more details.

1.4.3 Generic types

Built-in collection type objects support indexing:

Type	Description
<code>list[str]</code>	list of <code>str</code> objects
<code>tuple[int, int]</code>	tuple of two <code>int</code> objects (<code>tuple[()]</code> is the empty tuple)
<code>tuple[int, ...]</code>	tuple of an arbitrary number of <code>int</code> objects
<code>dict[str, int]</code>	dictionary from <code>str</code> keys to <code>int</code> values
<code>Iterable[int]</code>	iterable object containing ints
<code>Sequence[bool]</code>	sequence of booleans (read-only)
<code>Mapping[str, int]</code>	mapping from <code>str</code> keys to <code>int</code> values (read-only)
<code>type[C]</code>	type object of <code>C</code> (<code>C</code> is a class/type variable/union of types)

The type `dict` is a *generic* class, signified by type arguments within `[...]`. For example, `dict[int, str]` is a dictionary from integers to strings and `dict[Any, Any]` is a dictionary of dynamically typed (arbitrary) values and keys. `list` is another generic class.

`Iterable`, `Sequence`, and `Mapping` are generic types that correspond to Python protocols. For example, a `str` object or a `list[str]` object is valid when `Iterable[str]` or `Sequence[str]` is expected. You can import them from `collections.abc` instead of importing from `typing`.

See *Using generic builtins* for more details.

These legacy types defined in `typing` are also supported:

Type	Description
<code>List[str]</code>	list of <code>str</code> objects
<code>Tuple[int, int]</code>	tuple of two <code>int</code> objects (<code>Tuple[()]</code> is the empty tuple)
<code>Tuple[int, ...]</code>	tuple of an arbitrary number of <code>int</code> objects
<code>Dict[str, int]</code>	dictionary from <code>str</code> keys to <code>int</code> values
<code>Type[C]</code>	type object of <code>C</code> (<code>C</code> is a class/type variable/union of types)

1.5 Type inference and type annotations

1.5.1 Type inference

For most variables, if you do not explicitly specify its type, mypy will infer the correct type based on what is initially assigned to the variable.

```
# Mypy will infer the type of these variables, despite no annotations
i = 1
reveal_type(i)  # Revealed type is "builtins.int"
l = [1, 2]
reveal_type(l)  # Revealed type is "builtins.list[builtins.int]"
```

Note

Note that mypy will not use type inference in dynamically typed functions (those without a function type annotation) — every local variable type defaults to `Any` in such functions. For more details, see *Dynamically typed code*.

```
def untyped_function():
    i = 1
    reveal_type(i)  # Revealed type is "Any"
                   # 'reveal_type' always outputs 'Any' in unchecked functions
```

1.5.2 Explicit types for variables

You can override the inferred type of a variable by using a variable type annotation:

```
x: int | str = 1
```

Without the type annotation, the type of `x` would be just `int`. We use an annotation to give it a more general type `int | str` (this type means that the value can be either an `int` or a `str`).

The best way to think about this is that the type annotation sets the type of the variable, not the type of the expression. For instance, mypy will complain about the following code:

```
x: int | str = 1.1  # error: Incompatible types in assignment
                   # (expression has type "float", variable has type "int | str")
```

Note

To explicitly override the type of an expression you can use `cast(<type>, <expression>)`. See *Casts* for details.

Note that you can explicitly declare the type of a variable without giving it an initial value:

```
# We only unpack two values, so there's no right-hand side value
# for mypy to infer the type of "cs" from:
a, b, *cs = 1, 2 # error: Need type annotation for "cs"

rs: list[int] # no assignment!
p, q, *rs = 1, 2 # OK
```

1.5.3 Explicit types for collections

The type checker cannot always infer the type of a list or a dictionary. This often arises when creating an empty list or dictionary and assigning it to a new variable that doesn't have an explicit variable type. Here is an example where mypy can't infer the type without some help:

```
l = [] # Error: Need type annotation for "l"
```

In these cases you can give the type explicitly using a type annotation:

```
l: list[int] = [] # Create empty list of int
d: dict[str, int] = {} # Create empty dictionary (str -> int)
```

1.5.4 Compatibility of container types

A quick note: container types can sometimes be unintuitive. We'll discuss this more in *Invariance vs covariance*. For example, the following program generates a mypy error, because mypy treats `list[int]` as incompatible with `list[object]`:

```
def f(l: list[object], k: list[int]) -> None:
    l = k # error: Incompatible types in assignment
```

The reason why the above assignment is disallowed is that allowing the assignment could result in non-int values stored in a list of `int`:

```
def f(l: list[object], k: list[int]) -> None:
    l = k
    l.append('x')
    print(k[-1]) # Ouch; a string in list[int]
```

Other container types like `dict` and `set` behave similarly.

You can still run the above program; it prints `x`. This illustrates the fact that static types do not affect the runtime behavior of programs. You can run programs with type check failures, which is often very handy when performing a large refactoring. Thus you can always 'work around' the type system, and it doesn't really limit what you can do in your program.

1.5.5 Context in type inference

Type inference is *bidirectional* and takes context into account.

Mypy will take into account the type of the variable on the left-hand side of an assignment when inferring the type of the expression on the right-hand side. For example, the following will type check:

```
def f(l: list[object]) -> None:
    l = [1, 2] # Infer type list[object] for [1, 2], not list[int]
```

The value expression `[1, 2]` is type checked with the additional context that it is being assigned to a variable of type `list[object]`. This is used to infer the type of the *expression* as `list[object]`.

Declared argument types are also used for type context. In this program mypy knows that the empty list `[]` should have type `list[int]` based on the declared type of `arg` in `foo`:

```
def foo(arg: list[int]) -> None:
    print('Items:', ', '.join(str(a) for a in arg))

foo([]) # OK
```

However, context only works within a single statement. Here mypy requires an annotation for the empty list, since the context would only be available in the following statement:

```
def foo(arg: list[int]) -> None:
    print('Items:', ', '.join(str(a) for a in arg))

a = [] # Error: Need type annotation for "a"
foo(a)
```

Working around the issue is easy by adding a type annotation:

```
...
a: list[int] = [] # OK
foo(a)
```

1.5.6 Silencing type errors

You might want to disable type checking on specific lines, or within specific files in your codebase. To do that, you can use a `# type: ignore` comment.

For example, say in its latest update, the web framework you use can now take an integer argument to `run()`, which starts it on localhost on that port. Like so:

```
# Starting app on http://localhost:8000
app.run(8000)
```

However, the devs forgot to update their type annotations for `run`, so mypy still thinks `run` only expects `str` types. This would give you the following error:

```
error: Argument 1 to "run" of "A" has incompatible type "int"; expected "str"
```

If you cannot directly fix the web framework yourself, you can temporarily disable type checking on that line, by adding a `# type: ignore`:

```
# Starting app on http://localhost:8000
app.run(8000) # type: ignore
```

This will suppress any mypy errors that would have raised on that specific line.

You should probably add some more information on the `# type: ignore` comment, to explain why the ignore was added in the first place. This could be a link to an issue on the repository responsible for the type stubs, or it could be a short explanation of the bug. To do that, use this format:

```
# Starting app on http://localhost:8000
app.run(8000) # type: ignore # `run()` in v2.0 accepts an `int`, as a port
```

Type ignore error codes

By default, mypy displays an error code for each error:

```
error: "str" has no attribute "trim" [attr-defined]
```

It is possible to add a specific error-code in your ignore comment (e.g. `# type: ignore[attr-defined]`) to clarify what's being silenced. You can find more information about error codes [here](#).

Other ways to silence errors

You can get mypy to silence errors about a specific variable by dynamically typing it with `Any`. See [Dynamically typed code](#) for more information.

```
from typing import Any

def f(x: Any, y: str) -> None:
    x = 'hello'
    x += 1 # OK
```

You can ignore all mypy errors in a file by adding a `# mypy: ignore-errors` at the top of the file:

```
# mypy: ignore-errors
# This is a test file, skipping type checking in it.
import unittest
...
```

You can also specify per-module configuration options in your *The mypy configuration file*. For example:

```
# Don't report errors in the 'package_to_fix_later' package
[mypy-package_to_fix_later.*]
ignore_errors = True

# Disable specific error codes in the 'tests' package
# Also don't require type annotations
[mypy-tests.*]
disable_error_code = var-annotated, has-type
allow_untyped_defs = True

# Silence import errors from the 'library_missing_types' package
[mypy-library_missing_types.*]
ignore_missing_imports = True
```

Finally, adding a `@typing.no_type_check` decorator to a class, method or function causes mypy to avoid type checking that class, method or function and to treat it as not having any type annotations.

```
@typing.no_type_check
def foo() -> str:
    return 12345 # No error!
```

1.6 Kinds of types

We’ve mostly restricted ourselves to built-in types until now. This section introduces several additional kinds of types. You are likely to need at least some of them to type check any non-trivial programs.

1.6.1 Class types

Every class is also a valid type. Any instance of a subclass is also compatible with all superclasses – it follows that every value is compatible with the `object` type (and incidentally also the `Any` type, discussed below). Mypy analyzes the bodies of classes to determine which methods and attributes are available in instances. This example uses subclassing:

```
class A:
    def f(self) -> int: # Type of self inferred (A)
        return 2

class B(A):
    def f(self) -> int:
        return 3
    def g(self) -> int:
        return 4

def foo(a: A) -> None:
    print(a.f()) # 3
    a.g()        # Error: "A" has no attribute "g"

foo(B()) # OK (B is a subclass of A)
```

1.6.2 The Any type

A value with the `Any` type is dynamically typed. Mypy doesn’t know anything about the possible runtime types of such value. Any operations are permitted on the value, and the operations are only checked at runtime. You can use `Any` as an “escape hatch” when you can’t use a more precise type for some reason.

This should not be confused with the `object` type, which represents the set of all values. Unlike `object`, `Any` introduces type unsafety — see *Any vs. object* for more.

`Any` is compatible with every other type, and vice versa. You can freely assign a value of type `Any` to a variable with a more precise type:

```
a: Any = None
s: str = ''
a = 2    # OK (assign "int" to "Any")
s = a    # OK (assign "Any" to "str")
```

Declared (and inferred) types are ignored (or *erased*) at runtime. They are basically treated as comments, and thus the above code does not generate a runtime error, even though `s` gets an `int` value when the program is run, while the declared type of `s` is actually `str`! You need to be careful with `Any` types, since they let you lie to mypy, and this could easily hide bugs.

If you do not define a function return value or argument types, these default to `Any`:

```
def show_heading(s) -> None:
    print('=== ' + s + ' ===') # No static type checking, as s has type Any

show_heading(1) # OK (runtime error only; mypy won't generate an error)
```

You should give a statically typed function an explicit `None` return type even if it doesn't return a value, as this lets mypy catch additional type errors:

```
def wait(t: float): # Implicit Any return value
    print('Waiting...')
    time.sleep(t)

if wait(2) > 1: # Mypy doesn't catch this error!
    ...
```

If we had used an explicit `None` return type, mypy would have caught the error:

```
def wait(t: float) -> None:
    print('Waiting...')
    time.sleep(t)

if wait(2) > 1: # Error: can't compare None and int
    ...
```

The `Any` type is discussed in more detail in section *Dynamically typed code*.

Note

A function without any types in the signature is dynamically typed. The body of a dynamically typed function is not checked statically, and local variables have implicit `Any` types. This makes it easier to migrate legacy Python code to mypy, as mypy won't complain about dynamically typed functions.

1.6.3 Tuple types

The type `tuple[T1, ..., Tn]` represents a tuple with the item types `T1, ..., Tn`:

```
# Use `typing.Tuple` in Python 3.8 and earlier
def f(t: tuple[int, str]) -> None:
    t = 1, 'foo' # OK
    t = 'foo', 1 # Type check error
```

A tuple type of this kind has exactly a specific number of items (2 in the above example). Tuples can also be used as immutable, varying-length sequences. You can use the type `tuple[T, ...]` (with a literal `...` – it's part of the syntax) for this purpose. Example:

```
def print_squared(t: tuple[int, ...]) -> None:
    for n in t:
        print(n, n ** 2)

print_squared(()) # OK
print_squared((1, 3, 5)) # OK
print_squared([1, 2]) # Error: only a tuple is valid
```

Note

Usually it's a better idea to use `Sequence[T]` instead of `tuple[T, ...]`, as `Sequence` is also compatible with lists and other non-tuple sequences.

Note

`tuple[...]` is valid as a base class in Python 3.6 and later, and always in stub files. In earlier Python versions you can sometimes work around this limitation by using a named tuple as a base class (see section *Named tuples*).

1.6.4 Callable types (and lambdas)

You can pass around function objects and bound methods in statically typed code. The type of a function that accepts arguments `A1, ..., An` and returns `Rt` is `Callable[[A1, ..., An], Rt]`. Example:

```
from collections.abc import Callable

def twice(i: int, next: Callable[[int], int]) -> int:
    return next(next(i))

def add(i: int) -> int:
    return i + 1

print(twice(3, add))    # 5
```

Note

Import `Callable[...]` from `typing` instead of `collections.abc` if you use Python 3.8 or earlier.

You can only have positional arguments, and only ones without default values, in callable types. These cover the vast majority of uses of callable types, but sometimes this isn't quite enough. Mypy recognizes a special form `Callable[... , T]` (with a literal `...`) which can be used in less typical cases. It is compatible with arbitrary callable objects that return a type compatible with `T`, independent of the number, types or kinds of arguments. Mypy lets you call such callable values with arbitrary arguments, without any checking – in this respect they are treated similar to a `(*args: Any, **kwargs: Any)` function signature. Example:

```
from collections.abc import Callable

def arbitrary_call(f: Callable[... , int]) -> int:
    return f('x') + f(y=2)    # OK

arbitrary_call(ord)          # No static error, but fails at runtime
arbitrary_call(open)         # Error: does not return an int
arbitrary_call(1)            # Error: 'int' is not callable
```

In situations where more precise or complex types of callbacks are necessary one can use flexible *callback protocols*. Lambdas are also supported. The lambda argument and return value types cannot be given explicitly; they are always inferred based on context using bidirectional type inference:

```
l = map(lambda x: x + 1, [1, 2, 3])    # Infer x as int and l as list[int]
```

If you want to give the argument or return value types explicitly, use an ordinary, perhaps nested function definition. Callables can also be used against type objects, matching their `__init__` or `__new__` signature:

```
from collections.abc import Callable

class C:
    def __init__(self, app: str) -> None:
        pass

CallableType = Callable[[str], C]

def class_or_callable(arg: CallableType) -> None:
    inst = arg("my_app")
    reveal_type(inst)  # Revealed type is "C"
```

This is useful if you want `arg` to be either a `Callable` returning an instance of `C` or the type of `C` itself. This also works with *callback protocols*.

1.6.5 Union types

Python functions often accept values of two or more different types. You can use *overloading* to represent this, but union types are often more convenient.

Use `T1 | ... | Tn` to construct a union type. For example, if an argument has type `int | str`, both integers and strings are valid argument values.

You can use an `isinstance()` check to narrow down a union type to a more specific type:

```
def f(x: int | str) -> None:
    x + 1      # Error: str + int is not valid
    if isinstance(x, int):
        # Here type of x is int.
        x + 1      # OK
    else:
        # Here type of x is str.
        x + 'a'     # OK

f(1)      # OK
f('x')    # OK
f(1.1)    # Error
```

Note

Operations are valid for union types only if they are valid for *every* union item. This is why it's often necessary to use an `isinstance()` check to first narrow down a union type to a non-union type. This also means that it's recommended to avoid union types as function return types, since the caller may have to use `isinstance()` before doing anything interesting with the value.

Python 3.9 and older only partially support this syntax. Instead, you can use the legacy `Union[T1, ..., Tn]` type constructor. Example:

```
from typing import Union
```

(continues on next page)

(continued from previous page)

```
def f(x: Union[int, str]) -> None:
    ...
```

It is also possible to use the new syntax with versions of Python where it isn't supported by the runtime with some limitations, if you use `from __future__ import annotations` (see [Annotation issues at runtime](#)):

```
from __future__ import annotations

def f(x: int | str) -> None:    # OK on Python 3.7 and later
    ...
```

1.6.6 Optional types and the None type

You can use `T | None` to define a type variant that allows `None` values, such as `int | None`. This is called an *optional type*:

```
def strlen(s: str) -> int | None:
    if not s:
        return None    # OK
    return len(s)

def strlen_invalid(s: str) -> int:
    if not s:
        return None    # Error: None not compatible with int
    return len(s)
```

You can also use the `Optional` type modifier, such as `Optional[int]` (`Optional[X]` is the shorthand for `Union[X, None]`):

```
from typing import Optional

def strlen(s: str) -> Optional[int]:
    ...
```

Most operations will not be allowed on unguarded `None` or *optional* values (values with an optional type):

```
def my_inc(x: int | None) -> int:
    return x + 1    # Error: Cannot add None and int
```

Instead, an explicit `None` check is required. Mypy has powerful type inference that lets you use regular Python idioms to guard against `None` values. For example, mypy recognizes `is None` checks:

```
def my_inc(x: int | None) -> int:
    if x is None:
        return 0
    else:
        # The inferred type of x is just int here.
        return x + 1
```

Mypy will infer the type of `x` to be `int` in the `else` block due to the check against `None` in the `if` condition.

Other supported checks for guarding against a `None` value include `if x is not None`, `if x` and `if not x`. Additionally, mypy understands `None` checks within logical expressions:

```
def concat(x: str | None, y: str | None) -> str | None:
    if x is not None and y is not None:
        # Both x and y are not None here
        return x + y
    else:
        return None
```

Sometimes mypy doesn't realize that a value is never `None`. This notably happens when a class instance can exist in a partially defined state, where some attribute is initialized to `None` during object construction, but a method assumes that the attribute is no longer `None`. Mypy will complain about the possible `None` value. You can use `assert x is not None` to work around this in the method:

```
class Resource:
    path: str | None = None

    def initialize(self, path: str) -> None:
        self.path = path

    def read(self) -> str:
        # We require that the object has been initialized.
        assert self.path is not None
        with open(self.path) as f: # OK
            return f.read()

r = Resource()
r.initialize('/foo/bar')
r.read()
```

When initializing a variable as `None`, `None` is usually an empty place-holder value, and the actual value has a different type. This is why you need to annotate an attribute in cases like the class `Resource` above:

```
class Resource:
    path: str | None = None
    ...
```

This also works for attributes defined within methods:

```
class Counter:
    def __init__(self) -> None:
        self.count: int | None = None
```

Often it's easier to not use any initial value for an attribute. This way you don't need to use an optional type and can avoid `assert ... is not None` checks. No initial value is needed if you annotate an attribute in the class body:

```
class Container:
    items: list[str] # No initial value
```

Mypy generally uses the first assignment to a variable to infer the type of the variable. However, if you assign both a `None` value and a non-`None` value in the same scope, mypy can usually do the right thing without an annotation:

```
def f(i: int) -> None:
    n = None # Inferred type 'int | None' because of the assignment below
    if i > 0:
```

(continues on next page)

(continued from previous page)

```
n = i
...
```

Sometimes you may get the error “Cannot determine type of <something>”. In this case you should add an explicit `... | None` annotation.

Note

`None` is a type with only one value, `None`. `None` is also used as the return type for functions that don’t return a value, i.e. functions that implicitly return `None`.

Note

The Python interpreter internally uses the name `NoneType` for the type of `None`, but `None` is always used in type annotations. The latter is shorter and reads better. (`NoneType` is available as `types.NoneType` on Python 3.10+, but is not exposed at all on earlier versions of Python.)

Note

The type `Optional[T]` *does not* mean a function parameter with a default value. It simply means that `None` is a valid argument value. This is a common confusion because `None` is a common default value for parameters, and parameters with default values are sometimes called *optional* parameters (or arguments).

1.6.7 Type aliases

In certain situations, type names may end up being long and painful to type, especially if they are used frequently:

```
def f() -> list[dict[tuple[int, str], set[int]]] | tuple[str, list[str]]:
    ...
```

When cases like this arise, you can define a type alias by simply assigning the type to a variable (this is an *implicit type alias*):

```
AliasType = list[dict[tuple[int, str], set[int]]] | tuple[str, list[str]]

# Now we can use AliasType in place of the full name:

def f() -> AliasType:
    ...
```

Note

A type alias does not create a new type. It’s just a shorthand notation for another type – it’s equivalent to the target type except for *generic aliases*.

Python 3.12 introduced the `type` statement for defining *explicit type aliases*. Explicit type aliases are unambiguous and can also improve readability by making the intent clear:

```
type AliasType = list[dict[tuple[int, str], set[int]]] | tuple[str, list[str]]

# Now we can use AliasType in place of the full name:

def f() -> AliasType:
    ...
```

There can be confusion about exactly when an assignment defines an implicit type alias – for example, when the alias contains forward references, invalid types, or violates some other restrictions on type alias declarations. Because the distinction between an unannotated variable and a type alias is implicit, ambiguous or incorrect type alias declarations default to defining a normal variable instead of a type alias.

Aliases defined using the `type` statement have these properties, which distinguish them from implicit type aliases:

- The definition may contain forward references without having to use string literal escaping, since it is evaluated lazily.
- The alias can be used in type annotations, type arguments, and casts, but it can't be used in contexts which require a class object. For example, it's not valid as a base class and it can't be used to construct instances.

There is also use an older syntax for defining explicit type aliases ([PEP 613](#)):

```
from typing import TypeAlias

AliasType: TypeAlias = list[dict[tuple[int, str], set[int]]] | tuple[str, list[str]]
```

1.6.8 Named tuples

Mypy recognizes named tuples and can type check code that defines or uses them. In this example, we can detect code trying to access a missing attribute:

```
Point = namedtuple('Point', ['x', 'y'])
p = Point(x=1, y=2)
print(p.z) # Error: Point has no attribute 'z'
```

If you use `namedtuple` to define your named tuple, all the items are assumed to have `Any` types. That is, mypy doesn't know anything about item types. You can use `NamedTuple` to also define item types:

```
from typing import NamedTuple

Point = NamedTuple('Point', [('x', int),
                              ('y', int)])
p = Point(x=1, y='x') # Argument has incompatible type "str"; expected "int"
```

Python 3.6 introduced an alternative, class-based syntax for named tuples with types:

```
from typing import NamedTuple

class Point(NamedTuple):
    x: int
    y: int

p = Point(x=1, y='x') # Argument has incompatible type "str"; expected "int"
```

Note

You can use the raw `NamedTuple` “pseudo-class” in type annotations if any `NamedTuple` object is valid.

For example, it can be useful for deserialization:

```
def deserialize_named_tuple(arg: NamedTuple) -> Dict[str, Any]:
    return arg._asdict()

Point = namedtuple('Point', ['x', 'y'])
Person = NamedTuple('Person', [('name', str), ('age', int)])

deserialize_named_tuple(Point(x=1, y=2)) # ok
deserialize_named_tuple(Person(name='Nikita', age=18)) # ok

# Error: Argument 1 to "deserialize_named_tuple" has incompatible type
# "Tuple[int, int]"; expected "NamedTuple"
deserialize_named_tuple((1, 2))
```

Note that this behavior is highly experimental, non-standard, and may not be supported by other type checkers and IDEs.

1.6.9 The type of class objects

(Freely after [PEP 484: The type of class objects](#).)

Sometimes you want to talk about class objects that inherit from a given class. This can be spelled as `type[C]` (or, on Python 3.8 and lower, `typing.Type[C]`) where `C` is a class. In other words, when `C` is the name of a class, using `C` to annotate an argument declares that the argument is an instance of `C` (or of a subclass of `C`), but using `type[C]` as an argument annotation declares that the argument is a class object deriving from `C` (or `C` itself).

For example, assume the following classes:

```
class User:
    # Defines fields like name, email

class BasicUser(User):
    def upgrade(self):
        """Upgrade to Pro"""

class ProUser(User):
    def pay(self):
        """Pay bill"""
```

Note that `ProUser` doesn't inherit from `BasicUser`.

Here's a function that creates an instance of one of these classes if you pass it the right class object:

```
def new_user(user_class):
    user = user_class()
    # (Here we could write the user object to a database)
    return user
```

How would we annotate this function? Without the ability to parameterize `type`, the best we could do would be:

```
def new_user(user_class: type) -> User:
    # Same implementation as before
```

This seems reasonable, except that in the following example, mypy doesn't see that the `buyer` variable has type `ProUser`:

```
buyer = new_user(ProUser)
buyer.pay() # Rejected, not a method on User
```

However, using the `type[C]` syntax and a type variable with an upper bound (see *Type variables with upper bounds*) we can do better (Python 3.12 syntax):

```
def new_user[U: User](user_class: type[U]) -> U:
    # Same implementation as before
```

Here is the example using the legacy syntax (Python 3.11 and earlier):

```
U = TypeVar('U', bound=User)

def new_user(user_class: type[U]) -> U:
    # Same implementation as before
```

Now mypy will infer the correct type of the result when we call `new_user()` with a specific subclass of `User`:

```
beginner = new_user(BasicUser) # Inferred type is BasicUser
beginner.upgrade() # OK
```

Note

The value corresponding to `type[C]` must be an actual class object that's a subtype of `C`. Its constructor must be compatible with the constructor of `C`. If `C` is a type variable, its upper bound must be a class object.

For more details about `type[]` and `typing.Type[]`, see [PEP 484: The type of class objects](#).

1.6.10 Generators

A basic generator that only yields values can be succinctly annotated as having a return type of either `Iterator[YieldType]` or `Iterable[YieldType]`. For example:

```
def squares(n: int) -> Iterator[int]:
    for i in range(n):
        yield i * i
```

A good rule of thumb is to annotate functions with the most specific return type possible. However, you should also take care to avoid leaking implementation details into a function's public API. In keeping with these two principles, prefer `Iterator[YieldType]` over `Iterable[YieldType]` as the return-type annotation for a generator function, as it lets mypy know that users are able to call `next()` on the object returned by the function. Nonetheless, bear in mind that `Iterable` may sometimes be the better option, if you consider it an implementation detail that `next()` can be called on the object returned by your function.

If you want your generator to accept values via the `send()` method or return a value, on the other hand, you should use the `Generator[YieldType, SendType, ReturnType]` generic type instead of either `Iterator` or `Iterable`. For example:

```
def echo_round() -> Generator[int, float, str]:
    sent = yield 0
    while sent >= 0:
        sent = yield round(sent)
    return 'Done'
```

Note that unlike many other generics in the typing module, the `SendType` of `Generator` behaves contravariantly, not covariantly or invariantly.

If you do not plan on receiving or returning values, then set the `SendType` or `ReturnType` to `None`, as appropriate. For example, we could have annotated the first example as the following:

```
def squares(n: int) -> Generator[int, None, None]:
    for i in range(n):
        yield i * i
```

This is slightly different from using `Iterator[int]` or `Iterable[int]`, since generators have `close()`, `send()`, and `throw()` methods that generic iterators and iterables don't. If you plan to call these methods on the returned generator, use the `Generator` type instead of `Iterator` or `Iterable`.

1.7 Class basics

This section will help get you started annotating your classes. Built-in classes such as `int` also follow these same rules.

1.7.1 Instance and class attributes

The mypy type checker detects if you are trying to access a missing attribute, which is a very common programming error. For this to work correctly, instance and class attributes must be defined or initialized within the class. Mypy infers the types of attributes:

```
class A:
    def __init__(self, x: int) -> None:
        self.x = x # Aha, attribute 'x' of type 'int'

a = A(1)
a.x = 2 # OK!
a.y = 3 # Error: "A" has no attribute "y"
```

This is a bit like each class having an implicitly defined `__slots__` attribute. This is only enforced during type checking and not when your program is running.

You can declare types of variables in the class body explicitly using a type annotation:

```
class A:
    x: list[int] # Declare attribute 'x' of type list[int]

a = A()
a.x = [1] # OK
```

As in Python generally, a variable defined in the class body can be used as a class or an instance variable. (As discussed in the next section, you can override this with a `ClassVar` annotation.)

Similarly, you can give explicit types to instance variables defined in a method:

```
class A:
    def __init__(self) -> None:
        self.x: list[int] = []

    def f(self) -> None:
        self.y: Any = 0
```

You can only define an instance variable within a method if you assign to it explicitly using `self`:

```
class A:
    def __init__(self) -> None:
        self.y = 1    # Define 'y'
        a = self
        a.x = 1       # Error: 'x' not defined
```

1.7.2 Annotating `__init__` methods

The `__init__` method is somewhat special – it doesn’t return a value. This is best expressed as `-> None`. However, since many feel this is redundant, it is allowed to omit the return type declaration on `__init__` methods **if at least one argument is annotated**. For example, in the following classes `__init__` is considered fully annotated:

```
class C1:
    def __init__(self) -> None:
        self.var = 42

class C2:
    def __init__(self, arg: int):
        self.var = arg
```

However, if `__init__` has no annotated arguments and no return type annotation, it is considered an untyped method:

```
class C3:
    def __init__(self):
        # This body is not type checked
        self.var = 42 + 'abc'
```

1.7.3 Class attribute annotations

You can use a `ClassVar[t]` annotation to explicitly declare that a particular attribute should not be set on instances:

```
from typing import ClassVar

class A:
    x: ClassVar[int] = 0 # Class variable only

A.x += 1 # OK

a = A()
a.x = 1 # Error: Cannot assign to class variable "x" via instance
print(a.x) # OK -- can be read through an instance
```

It’s not necessary to annotate all class variables using `ClassVar`. An attribute without the `ClassVar` annotation can still be used as a class variable. However, mypy won’t prevent it from being used as an instance variable, as discussed previously:

```
class A:
    x = 0 # Can be used as a class or instance variable

A.x += 1 # OK

a = A()
a.x = 1 # Also OK
```

Note that `ClassVar` is not a class, and you can't use it with `isinstance()` or `issubclass()`. It does not change Python runtime behavior – it's only for type checkers such as mypy (and also helpful for human readers).

You can also omit the square brackets and the variable type in a `ClassVar` annotation, but this might not do what you'd expect:

```
class A:
    y: ClassVar = 0 # Type implicitly Any!
```

In this case the type of the attribute will be implicitly `Any`. This behavior will change in the future, since it's surprising.

An explicit `ClassVar` may be particularly handy to distinguish between class and instance variables with callable types. For example:

```
from collections.abc import Callable
from typing import ClassVar

class A:
    foo: Callable[[int], None]
    bar: ClassVar[Callable[[A, int], None]]
    bad: Callable[[A], None]

A().foo(42) # OK
A().bar(42) # OK
A().bad() # Error: Too few arguments
```

Note

A `ClassVar` type parameter cannot include type variables: `ClassVar[T]` and `ClassVar[list[T]]` are both invalid if `T` is a type variable (see [Defining generic classes](#) for more about type variables).

1.7.4 Overriding statically typed methods

When overriding a statically typed method, mypy checks that the override has a compatible signature:

```
class Base:
    def f(self, x: int) -> None:
        ...

class Derived1(Base):
    def f(self, x: str) -> None: # Error: type of 'x' incompatible
        ...

class Derived2(Base):
    def f(self, x: int, y: int) -> None: # Error: too many arguments
```

(continues on next page)

(continued from previous page)

```

...

class Derived3(Base):
    def f(self, x: int) -> None:    # OK
    ...

class Derived4(Base):
    def f(self, x: float) -> None:  # OK: mypy treats int as a subtype of float
    ...

class Derived5(Base):
    def f(self, x: int, y: int = 0) -> None:  # OK: accepts more than the base
                                              # class method
    ...

```

Note

You can also vary return types **covariantly** in overriding. For example, you could override the return type `Iterable[int]` with a subtype such as `list[int]`. Similarly, you can vary argument types **contravariantly** – subclasses can have more general argument types.

In order to ensure that your code remains correct when renaming methods, it can be helpful to explicitly mark a method as overriding a base method. This can be done with the `@override` decorator. `@override` can be imported from `typing` starting with Python 3.12 or from `typing_extensions` for use with older Python versions. If the base method is then renamed while the overriding method is not, mypy will show an error:

```

from typing import override

class Base:
    def f(self, x: int) -> None:
    ...
    def g_renamed(self, y: str) -> None:
    ...

class Derived1(Base):
    @override
    def f(self, x: int) -> None:    # OK
    ...

    @override
    def g(self, y: str) -> None:    # Error: no corresponding base method found
    ...

```

Note

Use `--enable-error-code explicit-override` to require that method overrides use the `@override` decorator. Emit an error if it is missing.

You can also override a statically typed method with a dynamically typed one. This allows dynamically typed code to override methods defined in library classes without worrying about their type signatures.

As always, relying on dynamically typed code can be unsafe. There is no runtime enforcement that the method override returns a value that is compatible with the original return type, since annotations have no effect at runtime:

```
class Base:
    def inc(self, x: int) -> int:
        return x + 1

class Derived(Base):
    def inc(self, x):      # Override, dynamically typed
        return 'hello'   # Incompatible with 'Base', but no mypy error
```

1.7.5 Abstract base classes and multiple inheritance

Mypy supports Python [abstract base classes](#) (ABCs). Abstract classes have at least one abstract method or property that must be implemented by any *concrete* (non-abstract) subclass. You can define abstract base classes using the `abc.ABCMeta` metaclass and the `@abc.abstractmethod` function decorator. Example:

```
from abc import ABCMeta, abstractmethod

class Animal(metaclass=ABCMeta):
    @abstractmethod
    def eat(self, food: str) -> None: pass

    @property
    @abstractmethod
    def can_walk(self) -> bool: pass

class Cat(Animal):
    def eat(self, food: str) -> None:
        ... # Body omitted

    @property
    def can_walk(self) -> bool:
        return True

x = Animal() # Error: 'Animal' is abstract due to 'eat' and 'can_walk'
y = Cat()    # OK
```

Note that mypy performs checking for unimplemented abstract methods even if you omit the `ABCMeta` metaclass. This can be useful if the metaclass would cause runtime metaclass conflicts.

Since you can't create instances of ABCs, they are most commonly used in type annotations. For example, this method accepts arbitrary iterables containing arbitrary animals (instances of concrete `Animal` subclasses):

```
def feed_all(animals: Iterable[Animal], food: str) -> None:
    for animal in animals:
        animal.eat(food)
```

There is one important peculiarity about how ABCs work in Python – whether a particular class is abstract or not is somewhat implicit. In the example below, `Derived` is treated as an abstract base class since `Derived` inherits an abstract method from `Base` and doesn't explicitly implement it. The definition of `Derived` generates no errors from mypy, since it's a valid ABC:

```

from abc import ABCMeta, abstractmethod

class Base(metaclass=ABCMeta):
    @abstractmethod
    def f(self, x: int) -> None: pass

class Derived(Base): # No error -- Derived is implicitly abstract
    def g(self) -> None:
        ...

```

Attempting to create an instance of `Derived` will be rejected, however:

```
d = Derived() # Error: 'Derived' is abstract
```

Note

It's a common error to forget to implement an abstract method. As shown above, the class definition will not generate an error in this case, but any attempt to construct an instance will be flagged as an error.

Mypy allows you to omit the body for an abstract method, but if you do so, it is unsafe to call such method via `super()`. For example:

```

from abc import abstractmethod

class Base:
    @abstractmethod
    def foo(self) -> int: pass
    @abstractmethod
    def bar(self) -> int:
        return 0
class Sub(Base):
    def foo(self) -> int:
        return super().foo() + 1 # error: Call to abstract method "foo" of "Base"
                                # with trivial body via super() is unsafe

    @abstractmethod
    def bar(self) -> int:
        return super().bar() + 1 # This is OK however.

```

A class can inherit any number of classes, both abstract and concrete. As with normal overrides, a dynamically typed method can override or implement a statically typed method defined in any base class, including an abstract method defined in an abstract base class.

You can implement an abstract property using either a normal property or an instance variable.

1.7.6 Slots

When a class has explicitly defined `__slots__`, mypy will check that all attributes assigned to are members of `__slots__`:

```

class Album:
    __slots__ = ('name', 'year')

    def __init__(self, name: str, year: int) -> None:

```

(continues on next page)

(continued from previous page)

```

    self.name = name
    self.year = year
    # Error: Trying to assign name "released" that is not in "__slots__" of type
    ↪ "Album"
    self.released = True

my_album = Album('Songs about Python', 2021)

```

Mypy will only check attribute assignments against `__slots__` when the following conditions hold:

1. All base classes (except builtin ones) must have explicit `__slots__` defined (this mirrors Python semantics).
2. `__slots__` does not include `__dict__`. If `__slots__` includes `__dict__`, arbitrary attributes can be set, similar to when `__slots__` is not defined (this mirrors Python semantics).
3. All values in `__slots__` must be string literals.

1.8 Annotation issues at runtime

Idiomatic use of type annotations can sometimes run up against what a given version of Python considers legal code. This section describes these scenarios and explains how to get your code running again. Generally speaking, we have three tools at our disposal:

- Use of string literal types or type comments
- Use of `typing.TYPE_CHECKING`
- Use of `from __future__ import annotations` (PEP 563)

We provide a description of these before moving onto discussion of specific problems you may encounter.

1.8.1 String literal types and type comments

Mypy lets you add type annotations using the (now deprecated) `# type:` type comment syntax. These were required with Python versions older than 3.6, since they didn't support type annotations on variables. Example:

```

a = 1 # type: int

def f(x): # type: (int) -> int
    return x + 1

# Alternative type comment syntax for functions with many arguments
def send_email(
    address,      # type: Union[str, List[str]]
    sender,       # type: str
    cc,           # type: Optional[List[str]]
    subject='',
    body=None     # type: List[str]
):
    # type: (...) -> bool

```

Type comments can't cause runtime errors because comments are not evaluated by Python.

In a similar way, using string literal types sidesteps the problem of annotations that would cause runtime errors.

Any type can be entered as a string literal, and you can combine string-literal types with non-string-literal types freely:

```
def f(a: list['A']) -> None: ... # OK, prevents NameError since A is defined later
def g(n: 'int') -> None: ...    # Also OK, though not useful

class A: pass
```

String literal types are never needed in `# type:` comments and *stub files*.

String literal types must be defined (or imported) later *in the same module*. They cannot be used to leave cross-module references unresolved. (For dealing with import cycles, see *Import cycles*.)

1.8.2 Future annotations import (PEP 563)

Many of the issues described here are caused by Python trying to evaluate annotations. Future Python versions (potentially Python 3.14) will by default no longer attempt to evaluate function and variable annotations. This behaviour is made available in Python 3.7 and later through the use of `from __future__ import annotations`.

This can be thought of as automatic string literal-ification of all function and variable annotations. Note that function and variable annotations are still required to be valid Python syntax. For more details, see [PEP 563](#).

Note

Even with the `__future__` import, there are some scenarios that could still require string literals or result in errors, typically involving use of forward references or generics in:

- *type aliases* not defined using the `type` statement;
- *type narrowing*;
- type definitions (see `TypeVar`, `NewType`, `NamedTuple`);
- base classes.

```
# base class example
from __future__ import annotations

class A(tuple['B', 'C']): ... # String literal types needed here
class B: ...
class C: ...
```

Warning

Some libraries may have use cases for dynamic evaluation of annotations, for instance, through use of `typing.get_type_hints` or `eval`. If your annotation would raise an error when evaluated (say by using [PEP 604](#) syntax with Python 3.9), you may need to be careful when using such libraries.

1.8.3 typing.TYPE_CHECKING

The `typing` module defines a `TYPE_CHECKING` constant that is `False` at runtime but treated as `True` while type checking.

Since code inside `if TYPE_CHECKING:` is not executed at runtime, it provides a convenient way to tell mypy something without the code being evaluated at runtime. This is most useful for resolving *import cycles*.

1.8.4 Class name forward references

Python does not allow references to a class object before the class is defined (aka forward reference). Thus this code does not work as expected:

```
def f(x: A) -> None: ... # NameError: name "A" is not defined
class A: ...
```

Starting from Python 3.7, you can add `from __future__ import annotations` to resolve this, as discussed earlier:

```
from __future__ import annotations

def f(x: A) -> None: ... # OK
class A: ...
```

For Python 3.6 and below, you can enter the type as a string literal or type comment:

```
def f(x: 'A') -> None: ... # OK

# Also OK
def g(x): # type: (A) -> None
    ...

class A: ...
```

Of course, instead of using future annotations import or string literal types, you could move the function definition after the class definition. This is not always desirable or even possible, though.

1.8.5 Import cycles

An import cycle occurs where module A imports module B and module B imports module A (perhaps indirectly, e.g. A -> B -> C -> A). Sometimes in order to add type annotations you have to add extra imports to a module and those imports cause cycles that didn't exist before. This can lead to errors at runtime like:

```
ImportError: cannot import name 'b' from partially initialized module 'A' (most likely
↳ due to a circular import)
```

If those cycles do become a problem when running your program, there's a trick: if the import is only needed for type annotations and you're using a) the *future annotations import*, or b) string literals or type comments for the relevant annotations, you can write the imports inside `if TYPE_CHECKING:` so that they are not executed at runtime. Example:

File `foo.py`:

```
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    import bar

def listify(arg: 'bar.BarClass') -> 'list[bar.BarClass]':
    return [arg]
```

File `bar.py`:

```
from foo import listify
```

(continues on next page)

(continued from previous page)

```
class BarClass:
    def listifyme(self) -> 'list[BarClass]':
        return listify(self)
```

1.8.6 Using classes that are generic in stubs but not at runtime

Some classes are declared as *generic* in stubs, but not at runtime.

In Python 3.8 and earlier, there are several examples within the standard library, for instance, `os.PathLike` and `queue.Queue`. Subscripting such a class will result in a runtime error:

```
from queue import Queue

class Tasks(Queue[str]): # TypeError: 'type' object is not subscriptable
    ...

results: Queue[int] = Queue() # TypeError: 'type' object is not subscriptable
```

To avoid errors from use of these generics in annotations, just use the *future annotations import* (or string literals or type comments for Python 3.6 and below).

To avoid errors when inheriting from these classes, things are a little more complicated and you need to use *typing.TYPE_CHECKING*:

```
from typing import TYPE_CHECKING
from queue import Queue

if TYPE_CHECKING:
    BaseQueue = Queue[str] # this is only processed by mypy
else:
    BaseQueue = Queue # this is not seen by mypy but will be executed at runtime

class Tasks(BaseQueue): # OK
    ...

task_queue: Tasks
reveal_type(task_queue.get()) # Reveals str
```

If your subclass is also generic, you can use the following (using the legacy syntax for generic classes):

```
from typing import TYPE_CHECKING, TypeVar, Generic
from queue import Queue

_T = TypeVar("_T")
if TYPE_CHECKING:
    class _MyQueueBase(Queue[_T]): pass
else:
    class _MyQueueBase(Generic[_T], Queue): pass

class MyQueue(_MyQueueBase[_T]): pass

task_queue: MyQueue[str]
reveal_type(task_queue.get()) # Reveals str
```

In Python 3.9 and later, we can just inherit directly from `Queue[str]` or `Queue[T]` since its `queue.Queue` implements `__class_getitem__()`, so the class object can be subscripted at runtime. You may still encounter issues (even if you use a recent Python version) when subclassing generic classes defined in third-party libraries if types are generic only in stubs.

1.8.7 Using types defined in stubs but not at runtime

Sometimes stubs that you're using may define types you wish to reuse that do not exist at runtime. Importing these types naively will cause your code to fail at runtime with `ImportError` or `ModuleNotFoundError`. Similar to previous sections, these can be dealt with by using `typing.TYPE_CHECKING`:

```
from __future__ import annotations
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from _typeshed import SupportsRichComparison

def f(x: SupportsRichComparison) -> None
```

The `from __future__ import annotations` is required to avoid a `NameError` when using the imported symbol. For more information and caveats, see the section on *future annotations*.

1.8.8 Using generic builtins

Starting with Python 3.9 ([PEP 585](#)), the type objects of many collections in the standard library support subscription at runtime. This means that you no longer have to import the equivalents from `typing`; you can simply use the built-in collections or those from `collections.abc`:

```
from collections.abc import Sequence
x: list[str]
y: dict[int, str]
z: Sequence[str] = x
```

There is limited support for using this syntax in Python 3.7 and later as well: if you use `from __future__ import annotations`, mypy will understand this syntax in annotations. However, since this will not be supported by the Python interpreter at runtime, make sure you're aware of the caveats mentioned in the notes at *future annotations import*.

1.8.9 Using X | Y syntax for Unions

Starting with Python 3.10 ([PEP 604](#)), you can spell union types as `x: int | str`, instead of `x: typing.Union[int, str]`.

There is limited support for using this syntax in Python 3.7 and later as well: if you use `from __future__ import annotations`, mypy will understand this syntax in annotations, string literal types, type comments and stub files. However, since this will not be supported by the Python interpreter at runtime (if evaluated, `int | str` will raise `TypeError: unsupported operand type(s) for |: 'type' and 'type'`), make sure you're aware of the caveats mentioned in the notes at *future annotations import*.

1.8.10 Using new additions to the typing module

You may find yourself wanting to use features added to the `typing` module in earlier versions of Python than the addition.

The easiest way to do this is to install and use the `typing_extensions` package from PyPI for the relevant imports, for example:

```
from typing_extensions import TypeIs
```

If you don't want to rely on `typing_extensions` being installed on newer Pythons, you could alternatively use:

```
import sys
if sys.version_info >= (3, 13):
    from typing import TypeIs
else:
    from typing_extensions import TypeIs
```

This plays nicely well with following [PEP 508](#) dependency specification: `typing_extensions; python_version<"3.13"`

1.9 Protocols and structural subtyping

The Python type system supports two ways of deciding whether two objects are compatible as types: nominal subtyping and structural subtyping.

Nominal subtyping is strictly based on the class hierarchy. If class `Dog` inherits class `Animal`, it's a subtype of `Animal`. Instances of `Dog` can be used when `Animal` instances are expected. This form of subtyping is what Python's type system predominantly uses: it's easy to understand and produces clear and concise error messages, and matches how the native `isinstance` check works – based on class hierarchy.

Structural subtyping is based on the operations that can be performed with an object. Class `Dog` is a structural subtype of class `Animal` if the former has all attributes and methods of the latter, and with compatible types.

Structural subtyping can be seen as a static equivalent of duck typing, which is well known to Python programmers. See [PEP 544](#) for the detailed specification of protocols and structural subtyping in Python.

1.9.1 Predefined protocols

The `collections.abc`, `typing` and other stdlib modules define various protocol classes that correspond to common Python protocols, such as `Iterable[T]`. If a class defines a suitable `__iter__` method, mypy understands that it implements the iterable protocol and is compatible with `Iterable[T]`. For example, `IntList` below is iterable, over `int` values:

```
from __future__ import annotations

from collections.abc import Iterator, Iterable

class IntList:
    def __init__(self, value: int, next: IntList | None) -> None:
        self.value = value
        self.next = next

    def __iter__(self) -> Iterator[int]:
        current = self
        while current:
            yield current.value
            current = current.next

def print_numbered(items: Iterable[int]) -> None:
    for n, x in enumerate(items):
        print(n + 1, x)
```

(continues on next page)

(continued from previous page)

```
x = IntList(3, IntList(5, None))
print_numbered(x) # OK
print_numbered([4, 5]) # Also OK
```

Predefined protocol reference lists various protocols defined in `collections.abc` and `typing` and the signatures of the corresponding methods you need to define to implement each protocol.

Note

`typing` also contains deprecated aliases to protocols and ABCs defined in `collections.abc`, such as `Iterable[T]`.

1.9.2 Simple user-defined protocols

You can define your own protocol class by inheriting the special `Protocol` class:

```
from collections.abc import Iterable
from typing import Protocol

class SupportsClose(Protocol):
    # Empty method body (explicit '...')
    def close(self) -> None: ...

class Resource: # No SupportsClose base class!

    def close(self) -> None:
        self.resource.release()

    # ... other methods ...

def close_all(items: Iterable[SupportsClose]) -> None:
    for item in items:
        item.close()

close_all([Resource(), open('some/file')]) # OK
```

`Resource` is a subtype of the `SupportsClose` protocol since it defines a compatible `close` method. Regular file objects returned by `open()` are similarly compatible with the protocol, as they support `close()`.

1.9.3 Defining subprotocols and subclassing protocols

You can also define subprotocols. Existing protocols can be extended and merged using multiple inheritance. Example:

```
# ... continuing from the previous example

class SupportsRead(Protocol):
    def read(self, amount: int) -> bytes: ...

class TaggedReadableResource(SupportsClose, SupportsRead, Protocol):
    label: str
```

(continues on next page)

(continued from previous page)

```
class AdvancedResource(Resource):
    def __init__(self, label: str) -> None:
        self.label = label

    def read(self, amount: int) -> bytes:
        # some implementation
        ...

resource: TaggedReadableResource
resource = AdvancedResource('handle with care') # OK
```

Note that inheriting from an existing protocol does not automatically turn the subclass into a protocol – it just creates a regular (non-protocol) class or ABC that implements the given protocol (or protocols). The `Protocol` base class must always be explicitly present if you are defining a protocol:

```
class NotAProtocol(SupportsClose): # This is NOT a protocol
    new_attr: int

class Concrete:
    new_attr: int = 0

    def close(self) -> None:
        ...

# Error: nominal subtyping used by default
x: NotAProtocol = Concrete() # Error!
```

You can also include default implementations of methods in protocols. If you explicitly subclass these protocols you can inherit these default implementations.

Explicitly including a protocol as a base class is also a way of documenting that your class implements a particular protocol, and it forces mypy to verify that your class implementation is actually compatible with the protocol. In particular, omitting a value for an attribute or a method body will make it implicitly abstract:

```
class SomeProto(Protocol):
    attr: int # Note, no right hand side
    def method(self) -> str: ... # Literally just ... here

class ExplicitSubclass(SomeProto):
    pass

ExplicitSubclass() # error: Cannot instantiate abstract class 'ExplicitSubclass'
                  # with abstract attributes 'attr' and 'method'
```

Similarly, explicitly assigning to a protocol instance can be a way to ask the type checker to verify that your class implements a protocol:

```
_proto: SomeProto = cast(ExplicitSubclass, None)
```

1.9.4 Invariance of protocol attributes

A common issue with protocols is that protocol attributes are invariant. For example:

```
class Box(Protocol):
    content: object

class IntBox:
    content: int

def takes_box(box: Box) -> None: ...

takes_box(IntBox()) # error: Argument 1 to "takes_box" has incompatible type "IntBox";
                    # expected "Box"
                    # note: Following member(s) of "IntBox" have conflicts:
                    # note:     content: expected "object", got "int"
```

This is because Box defines content as a mutable attribute. Here's why this is problematic:

```
def takes_box_evil(box: Box) -> None:
    box.content = "asdf" # This is bad, since box.content is supposed to be an object

my_int_box = IntBox()
takes_box_evil(my_int_box)
my_int_box.content + 1 # Oops, TypeError!
```

This can be fixed by declaring content to be read-only in the Box protocol using @property:

```
class Box(Protocol):
    @property
    def content(self) -> object: ...

class IntBox:
    content: int

def takes_box(box: Box) -> None: ...

takes_box(IntBox(42)) # OK
```

1.9.5 Recursive protocols

Protocols can be recursive (self-referential) and mutually recursive. This is useful for declaring abstract recursive collections such as trees and linked lists:

```
from __future__ import annotations

from typing import Protocol

class TreeLike(Protocol):
    value: int

    @property
    def left(self) -> TreeLike | None: ...
```

(continues on next page)

(continued from previous page)

```

@property
def right(self) -> TreeLike | None: ...

class SimpleTree:
    def __init__(self, value: int) -> None:
        self.value = value
        self.left: SimpleTree | None = None
        self.right: SimpleTree | None = None

root: TreeLike = SimpleTree(0) # OK

```

1.9.6 Using `isinstance()` with protocols

You can use a protocol class with `isinstance()` if you decorate it with the `@runtime_checkable` class decorator. The decorator adds rudimentary support for runtime structural checks:

```

from typing import Protocol, runtime_checkable

@runtime_checkable
class Portable(Protocol):
    handles: int

class Mug:
    def __init__(self) -> None:
        self.handles = 1

def use(handles: int) -> None: ...

mug = Mug()
if isinstance(mug, Portable): # Works at runtime!
    use(mug.handles)

```

`isinstance()` also works with the *predefined protocols* in `typing` such as `Iterable`.

Warning

`isinstance()` with protocols is not completely safe at runtime. For example, signatures of methods are not checked. The runtime implementation only checks that all protocol members exist, not that they have the correct type. `issubclass()` with protocols will only check for the existence of methods.

Note

`isinstance()` with protocols can also be surprisingly slow. In many cases, you're better served by using `hasattr()` to check for the presence of attributes.

1.9.7 Callback protocols

Protocols can be used to define flexible callback types that are hard (or even impossible) to express using the `Callable[...]` syntax, such as variadic, overloaded, and complex generic callbacks. They are defined with a special `__call__` member:

```
from collections.abc import Iterable
from typing import Optional, Protocol

class Combiner(Protocol):
    def __call__(self, *vals: bytes, maxlen: int | None = None) -> list[bytes]: ...

def batch_proc(data: Iterable[bytes], cb_results: Combiner) -> bytes:
    for item in data:
        ...

def good_cb(*vals: bytes, maxlen: int | None = None) -> list[bytes]:
    ...
def bad_cb(*vals: bytes, maxitems: int | None) -> list[bytes]:
    ...

batch_proc([], good_cb)  # OK
batch_proc([], bad_cb)  # Error! Argument 2 has incompatible type because of
                        # different name and kind in the callback
```

Callback protocols and `Callable` types can be used mostly interchangeably. Parameter names in `__call__` methods must be identical, unless the parameters are positional-only. Example (using the legacy syntax for generic functions):

```
from collections.abc import Callable
from typing import Protocol, TypeVar

T = TypeVar('T')

class Copy(Protocol):
    # '/' marks the end of positional-only parameters
    def __call__(self, origin: T, /) -> T: ...

copy_a: Callable[[T], T]
copy_b: Copy

copy_a = copy_b  # OK
copy_b = copy_a  # Also OK
```

1.9.8 Binding of types in protocol attributes

All protocol attributes annotations are treated as externally visible types of those attributes. This means that for example callables are not bound, and descriptors are not invoked:

```
from typing import Callable, Protocol, overload

class Integer:
    @overload
    def __get__(self, instance: None, owner: object) -> Integer: ...
    @overload
    def __get__(self, instance: object, owner: object) -> int: ...
    # <some implementation>

class Example(Protocol):
    foo: Callable[[object], int]
    bar: Integer

ex: Example
reveal_type(ex.foo) # Revealed type is Callable[[object], int]
reveal_type(ex.bar) # Revealed type is Integer
```

In other words, protocol attribute types are handled as they would appear in a `self` attribute annotation in a regular class. If you want some protocol attributes to be handled as though they were defined at class level, you should declare them explicitly using `ClassVar[...]`. Continuing previous example:

```
from typing import ClassVar

class OtherExample(Protocol):
    # This style is *not recommended*, but may be needed to reuse
    # some complex callable types. Otherwise use regular methods.
    foo: ClassVar[Callable[[object], int]]
    # This may be needed to mimic descriptor access on Type[...] types,
    # otherwise use a plain "bar: int" style.
    bar: ClassVar[Integer]

ex2: OtherExample
reveal_type(ex2.foo) # Revealed type is Callable[[], int]
reveal_type(ex2.bar) # Revealed type is int
```

1.9.9 Predefined protocol reference

Iteration protocols

The iteration protocols are useful in many contexts. For example, they allow iteration of objects in `for` loops.

`collections.abc.Iterable[T]`

The *example above* has a simple implementation of an `__iter__` method.

```
def __iter__(self) -> Iterator[T]
```

See also `Iterable`.

collections.abc.Iterator[T]

```
def __next__(self) -> T
def __iter__(self) -> Iterator[T]
```

See also [Iterator](#).

Collection protocols

Many of these are implemented by built-in container types such as [list](#) and [dict](#), and these are also useful for user-defined collection objects.

collections.abc.Sized

This is a type for objects that support [len\(x\)](#).

```
def __len__(self) -> int
```

See also [Sized](#).

collections.abc.Container[T]

This is a type for objects that support the [in](#) operator.

```
def __contains__(self, x: object) -> bool
```

See also [Container](#).

collections.abc.Collection[T]

```
def __len__(self) -> int
def __iter__(self) -> Iterator[T]
def __contains__(self, x: object) -> bool
```

See also [Collection](#).

One-off protocols

These protocols are typically only useful with a single standard library function or class.

collections.abc.Reversible[T]

This is a type for objects that support [reversed\(x\)](#).

```
def __reversed__(self) -> Iterator[T]
```

See also [Reversible](#).

typing.SupportsAbs[T]

This is a type for objects that support [abs\(x\)](#). T is the type of value returned by [abs\(x\)](#).

```
def __abs__(self) -> T
```

See also [SupportsAbs](#).

typing.SupportsBytes

This is a type for objects that support `bytes(x)`.

```
def __bytes__(self) -> bytes
```

See also `SupportsBytes`.

typing.SupportsComplex

This is a type for objects that support `complex(x)`. Note that no arithmetic operations are supported.

```
def __complex__(self) -> complex
```

See also `SupportsComplex`.

typing.SupportsFloat

This is a type for objects that support `float(x)`. Note that no arithmetic operations are supported.

```
def __float__(self) -> float
```

See also `SupportsFloat`.

typing.SupportsInt

This is a type for objects that support `int(x)`. Note that no arithmetic operations are supported.

```
def __int__(self) -> int
```

See also `SupportsInt`.

typing.SupportsRound[T]

This is a type for objects that support `round(x)`.

```
def __round__(self) -> T
```

See also `SupportsRound`.

Async protocols

These protocols can be useful in async code. See *Typing async/await* for more information.

collections.abc.Awaitable[T]

```
def __await__(self) -> Generator[Any, None, T]
```

See also `Awaitable`.

collections.abc.AsyncIterable[T]

```
def __aiter__(self) -> AsyncIterator[T]
```

See also `AsyncIterable`.

collections.abc.AsyncIterator[T]

```
def __anext__(self) -> Awaitable[T]
def __aiter__(self) -> AsyncIterator[T]
```

See also `AsyncIterator`.

Context manager protocols

There are two protocols for context managers – one for regular context managers and one for async ones. These allow defining objects that can be used in `with` and `async with` statements.

contextlib.AbstractContextManager[T]

```
def __enter__(self) -> T
def __exit__(self,
             exc_type: type[BaseException] | None,
             exc_value: BaseException | None,
             traceback: TracebackType | None) -> bool | None
```

See also `AbstractContextManager`.

contextlib.AbstractAsyncContextManager[T]

```
def __aenter__(self) -> Awaitable[T]
def __aexit__(self,
             exc_type: type[BaseException] | None,
             exc_value: BaseException | None,
             traceback: TracebackType | None) -> Awaitable[bool | None]
```

See also `AbstractAsyncContextManager`.

1.10 Dynamically typed code

In *Dynamic vs static typing*, we discussed how bodies of functions that don't have any explicit type annotations in their function are “dynamically typed” and that mypy will not check them. In this section, we'll talk a little bit more about what that means and how you can enable dynamic typing on a more fine grained basis.

In cases where your code is too magical for mypy to understand, you can make a variable or parameter dynamically typed by explicitly giving it the type `Any`. Mypy will let you do basically anything with a value of type `Any`, including assigning a value of type `Any` to a variable of any type (or vice versa).

```
from typing import Any

num = 1          # Statically typed (inferred to be int)
num = 'x'        # error: Incompatible types in assignment (expression has type "str",
                 # ↪ variable has type "int")

dyn: Any = 1     # Dynamically typed (type Any)
dyn = 'x'       # OK

num = dyn        # No error, mypy will let you assign a value of type Any to any variable
num += 1         # Oops, mypy still thinks num is an int
```

You can think of `Any` as a way to locally disable type checking. See [Silencing type errors](#) for other ways you can shut up the type checker.

1.10.1 Operations on `Any` values

You can do anything using a value with type `Any`, and the type checker will not complain:

```
def f(x: Any) -> int:
    # All of these are valid!
    x.foobar(1, y=2)
    print(x[3] + 'f')
    if x:
        x.z = x(2)
    open(x).read()
    return x
```

Values derived from an `Any` value also usually have the type `Any` implicitly, as mypy can't infer a more precise result type. For example, if you get the attribute of an `Any` value or call a `Any` value the result is `Any`:

```
def f(x: Any) -> None:
    y = x.foo()
    reveal_type(y) # Revealed type is "Any"
    z = y.bar("mypy will let you do anything to y")
    reveal_type(z) # Revealed type is "Any"
```

`Any` types may propagate through your program, making type checking less effective, unless you are careful.

Function parameters without annotations are also implicitly `Any`:

```
def f(x) -> None:
    reveal_type(x) # Revealed type is "Any"
    x.can.do["anything", x]("wants", 2)
```

You can make mypy warn you about untyped function parameters using the `--disallow-untyped-defs` flag.

Generic types missing type parameters will have those parameters implicitly treated as `Any`:

```
def f(x: list) -> None:
    reveal_type(x) # Revealed type is "builtins.list[Any]"
    reveal_type(x[0]) # Revealed type is "Any"
    x[0].anything_goes() # OK
```

You can make mypy warn you about missing generic parameters using the `--disallow-any-generics` flag.

Finally, another major source of `Any` types leaking into your program is from third party libraries that mypy does not know about. This is particularly the case when using the `--ignore-missing-imports` flag. See [Missing imports](#) for more information about this.

1.10.2 `Any` vs. `object`

The type `object` is another type that can have an instance of arbitrary type as a value. Unlike `Any`, `object` is an ordinary static type (it is similar to `Object` in Java), and only operations valid for *all* types are accepted for `object` values. These are all valid:

```
def f(o: object) -> None:
    if o:
```

(continues on next page)

(continued from previous page)

```

    print(o)
    print(isinstance(o, int))
    o = 2
    o = 'foo'

```

These are, however, flagged as errors, since not all objects support these operations:

```

def f(o: object) -> None:
    o.foo()          # Error!
    o + 2            # Error!
    open(o)          # Error!
    n: int = 1
    n = o            # Error!

```

If you're not sure whether you need to use `object` or `Any`, use `object` – only switch to using `Any` if you get a type checker complaint.

You can use different *type narrowing* techniques to narrow `object` to a more specific type (subtype) such as `int`. Type narrowing is not needed with dynamically typed values (values with type `Any`).

1.11 Type narrowing

This section is dedicated to several type narrowing techniques which are supported by mypy.

Type narrowing is when you convince a type checker that a broader type is actually more specific, for instance, that an object of type `Shape` is actually of the narrower type `Square`.

The following type narrowing techniques are available:

- *Type narrowing expressions*
- *Casts*
- *User-Defined Type Guards*
- *TypeIs*

1.11.1 Type narrowing expressions

The simplest way to narrow a type is to use one of the supported expressions:

- `isinstance()` like in `isinstance(obj, float)` will narrow `obj` to have `float` type
- `issubclass()` like in `issubclass(cls, MyClass)` will narrow `cls` to be `Type[MyClass]`
- `type` like in `type(obj) is int` will narrow `obj` to have `int` type
- `callable()` like in `callable(obj)` will narrow `object` to callable type
- `obj is not None` will narrow `object` to its *non-optional form*

Type narrowing is contextual. For example, based on the condition, mypy will narrow an expression only within an `if` branch:

```

def function(arg: object):
    if isinstance(arg, int):
        # Type is narrowed within the `if` branch only
        reveal_type(arg) # Revealed type: "builtins.int"

```

(continues on next page)

(continued from previous page)

```

elif isinstance(arg, str) or isinstance(arg, bool):
    # Type is narrowed differently within this `elif` branch:
    reveal_type(arg) # Revealed type: "builtins.str | builtins.bool"

    # Subsequent narrowing operations will narrow the type further
    if isinstance(arg, bool):
        reveal_type(arg) # Revealed type: "builtins.bool"

# Back outside of the `if` statement, the type isn't narrowed:
reveal_type(arg) # Revealed type: "builtins.object"

```

Mypy understands the implications return or exception raising can have for what type an object could be:

```

def function(arg: int | str):
    if isinstance(arg, int):
        return

# `arg` can't be `int` at this point:
reveal_type(arg) # Revealed type: "builtins.str"

```

We can also use assert to narrow types in the same context:

```

def function(arg: Any):
    assert isinstance(arg, int)
    reveal_type(arg) # Revealed type: "builtins.int"

```

Note

With `--warn-unreachable` narrowing types to some impossible state will be treated as an error.

```

def function(arg: int):
    # error: Subclass of "int" and "str" cannot exist:
    # would have incompatible method signatures
    assert isinstance(arg, str)

    # error: Statement is unreachable
    print("so mypy concludes the assert will always trigger")

```

Without `--warn-unreachable` mypy will simply not check code it deems to be unreachable. See [Unreachable code](#) for more information.

```

x: int = 1
assert isinstance(x, str)
reveal_type(x) # Revealed type is "builtins.int"
print(x + '!') # Typechecks with `mypy`, but fails in runtime.

```

issubclass

Mypy can also use `issubclass()` for better type inference when working with types and metaclasses:

```

class MyCalcMeta(type):
    @classmethod

```

(continues on next page)

(continued from previous page)

```

def calc(cls) -> int:
    ...

def f(o: object) -> None:
    t = type(o) # We must use a variable here
    reveal_type(t) # Revealed type is "builtins.type"

    if issubclass(t, MyCalcMeta): # `issubclass(type(o), MyCalcMeta)` won't work
        reveal_type(t) # Revealed type is "Type[MyCalcMeta]"
        t.calc() # Okay

```

callable

Mypy knows what types are callable and which ones are not during type checking. So, we know what `callable()` will return. For example:

```

from collections.abc import Callable

x: Callable[[], int]

if callable(x):
    reveal_type(x) # N: Revealed type is "def () -> builtins.int"
else:
    ... # Will never be executed and will raise error with `--warn-unreachable`

```

The callable function can even split union types into callable and non-callable parts:

```

from collections.abc import Callable

x: int | Callable[[], int]

if callable(x):
    reveal_type(x) # N: Revealed type is "def () -> builtins.int"
else:
    reveal_type(x) # N: Revealed type is "builtins.int"

```

1.11.2 Casts

Mypy supports type casts that are usually used to coerce a statically typed value to a subtype. Unlike languages such as Java or C#, however, mypy casts are only used as hints for the type checker, and they don't perform a runtime type check. Use the function `cast()` to perform a cast:

```

from typing import cast

o: object = [1]
x = cast(list[int], o) # OK
y = cast(list[str], o) # OK (cast performs no actual runtime check)

```

To support runtime checking of casts such as the above, we'd have to check the types of all list items, which would be very inefficient for large lists. Casts are used to silence spurious type checker warnings and give the type checker a little help when it can't quite understand what is going on.

Note

You can use an assertion if you want to perform an actual runtime check:

```
def foo(o: object) -> None:
    print(o + 5)  # Error: can't add 'object' and 'int'
    assert isinstance(o, int)
    print(o + 5)  # OK: type of 'o' is 'int' here
```

You don't need a cast for expressions with type `Any`, or when assigning to a variable with type `Any`, as was explained earlier. You can also use `Any` as the cast target type – this lets you perform any operations on the result. For example:

```
from typing import cast, Any

x = 1
x.whatever()  # Type check error
y = cast(Any, x)
y.whatever()  # Type check OK (runtime error)
```

1.11.3 User-Defined Type Guards

Mypy supports User-Defined Type Guards ([PEP 647](#)).

A type guard is a way for programs to influence conditional type narrowing employed by a type checker based on runtime checks.

Basically, a `TypeGuard` is a “smart” alias for a `bool` type. Let's have a look at the regular `bool` example:

```
def is_str_list(val: list[object]) -> bool:
    """Determines whether all objects in the list are strings"""
    return all(isinstance(x, str) for x in val)

def func1(val: list[object]) -> None:
    if is_str_list(val):
        reveal_type(val)  # Reveals list[object]
        print(" ".join(val))  # Error: incompatible type
```

The same example with `TypeGuard`:

```
from typing import TypeGuard

def is_str_list(val: list[object]) -> TypeGuard[list[str]]:
    """Determines whether all objects in the list are strings"""
    return all(isinstance(x, str) for x in val)

def func1(val: list[object]) -> None:
    if is_str_list(val):
        reveal_type(val)  # list[str]
        print(" ".join(val))  # ok
```

How does it work? `TypeGuard` narrows the first function argument (`val`) to the type specified as the first type parameter (`list[str]`).

Note

Narrowing is `not` strict. For example, you can narrow `str` to `int`:

```
def f(value: str) -> TypeGuard[int]:
    return True
```

Note: since strict narrowing is not enforced, it's easy to break type safety.

However, there are many ways a determined or uninformed developer can subvert type safety – most commonly by using `cast` or `Any`. If a Python developer takes the time to learn about and implement user-defined type guards within their code, it is safe to assume that they are interested in type safety and will not write their type guard functions in a way that will undermine type safety or produce nonsensical results.

Generic TypeGuards

TypeGuard can also work with generic types (Python 3.12 syntax):

```
from typing import TypeGuard # use `typing_extensions` for `python<3.10`

def is_two_element_tuple[T](val: tuple[T, ...]) -> TypeGuard[tuple[T, T]]:
    return len(val) == 2

def func(names: tuple[str, ...]):
    if is_two_element_tuple(names):
        reveal_type(names) # tuple[str, str]
    else:
        reveal_type(names) # tuple[str, ...]
```

TypeGuards with parameters

Type guard functions can accept extra arguments (Python 3.12 syntax):

```
from typing import TypeGuard # use `typing_extensions` for `python<3.10`

def is_set_of[T](val: set[Any], type: type[T]) -> TypeGuard[set[T]]:
    return all(isinstance(x, type) for x in val)

items: set[Any]
if is_set_of(items, str):
    reveal_type(items) # set[str]
```

TypeGuards as methods

A method can also serve as a TypeGuard:

```
class StrValidator:
    def is_valid(self, instance: object) -> TypeGuard[str]:
        return isinstance(instance, str)

def func(to_validate: object) -> None:
    if StrValidator().is_valid(to_validate):
        reveal_type(to_validate) # Revealed type is "builtins.str"
```

Note

Note, that TypeGuard *does not narrow* types of `self` or `cls` implicit arguments.

If narrowing of `self` or `cls` is required, the value can be passed as an explicit argument to a type guard function:

```
class Parent:
    def method(self) -> None:
        reveal_type(self) # Revealed type is "Parent"
        if is_child(self):
            reveal_type(self) # Revealed type is "Child"

class Child(Parent):
    ...

def is_child(instance: Parent) -> TypeGuard[Child]:
    return isinstance(instance, Child)
```

Assignment expressions as TypeGuards

Sometimes you might need to create a new variable and narrow it to some specific type at the same time. This can be achieved by using TypeGuard together with `:=` operator.

```
from typing import TypeGuard # use `typing_extensions` for `python<3.10`

def is_float(a: object) -> TypeGuard[float]:
    return isinstance(a, float)

def main(a: object) -> None:
    if is_float(x := a):
        reveal_type(x) # N: Revealed type is 'builtins.float'
        reveal_type(a) # N: Revealed type is 'builtins.object'
    reveal_type(x) # N: Revealed type is 'builtins.object'
    reveal_type(a) # N: Revealed type is 'builtins.object'
```

What happens here?

1. We create a new variable `x` and assign a value of `a` to it
2. We run `is_float()` type guard on `x`
3. It narrows `x` to be `float` in the `if` context and does not touch `a`

Note

The same will work with `isinstance(x := a, float)` as well.

1.11.4 Typels

Mypy supports TypeIs ([PEP 742](#)).

A `TypeIs` *narrowing function* allows you to define custom type checks that can narrow the type of a variable in *both* the *if* and *else* branches of a conditional, similar to how the built-in `isinstance()` function works.

TypeIs is new in Python 3.13 — for use in older Python versions, use the backport from `typing_extensions`

Consider the following example using `TypeIs`:

```
from typing import TypeIs

def is_str(x: object) -> TypeIs[str]:
    return isinstance(x, str)

def process(x: int | str) -> None:
    if is_str(x):
        reveal_type(x)  # Revealed type is 'str'
        print(x.upper()) # Valid: x is str
    else:
        reveal_type(x)  # Revealed type is 'int'
        print(x + 1)    # Valid: x is int
```

In this example, the function `is_str` is a type narrowing function that returns `TypeIs[str]`. When used in an `if` statement, `x` is narrowed to `str` in the `if` branch and to `int` in the `else` branch.

Key points:

- The function must accept at least one positional argument.
- The return type is annotated as `TypeIs[T]`, where `T` is the type you want to narrow to.
- The function must return a `bool` value.
- In the `if` branch (when the function returns `True`), the type of the argument is narrowed to the intersection of its original type and `T`.
- In the `else` branch (when the function returns `False`), the type of the argument is narrowed to the intersection of its original type and the complement of `T`.

Typels vs TypeGuard

While both `TypeIs` and `TypeGuard` allow you to define custom type narrowing functions, they differ in important ways:

- **Type narrowing behavior:** `TypeIs` narrows the type in both the `if` and `else` branches, whereas `TypeGuard` narrows only in the `if` branch.
- **Compatibility requirement:** `TypeIs` requires that the narrowed type `T` be compatible with the input type of the function. `TypeGuard` does not have this restriction.
- **Type inference:** With `TypeIs`, the type checker may infer a more precise type by combining existing type information with `T`.

Here's an example demonstrating the behavior with `TypeGuard`:

```
from typing import TypeGuard, reveal_type

def is_str(x: object) -> TypeGuard[str]:
    return isinstance(x, str)

def process(x: int | str) -> None:
    if is_str(x):
        reveal_type(x)  # Revealed type is "builtins.str"
        print(x.upper()) # ok: x is str
    else:
        reveal_type(x)  # Revealed type is "Union[builtins.int, builtins.str]"
```

(continues on next page)

(continued from previous page)

```
print(x + 1) # ERROR: Unsupported operand types for + ("str" and "int")
↪[operator]
```

Generic Typels

TypeIs functions can also work with generic types:

```
from typing import TypeVar, TypeIs

T = TypeVar('T')

def is_two_element_tuple(val: tuple[T, ...]) -> TypeIs[tuple[T, T]]:
    return len(val) == 2

def process(names: tuple[str, ...]) -> None:
    if is_two_element_tuple(names):
        reveal_type(names) # Revealed type is 'tuple[str, str]'
    else:
        reveal_type(names) # Revealed type is 'tuple[str, ...]'
```

Typels with Additional Parameters

TypeIs functions can accept additional parameters beyond the first. The type narrowing applies only to the first argument.

```
from typing import Any, TypeVar, reveal_type, TypeIs

T = TypeVar('T')

def is_instance_of(val: Any, typ: type[T]) -> TypeIs[T]:
    return isinstance(val, typ)

def process(x: Any) -> None:
    if is_instance_of(x, int):
        reveal_type(x) # Revealed type is 'int'
        print(x + 1) # ok
    else:
        reveal_type(x) # Revealed type is 'Any'
```

Typels in Methods

A method can also serve as a TypeIs function. Note that in instance or class methods, the type narrowing applies to the second parameter (after self or cls).

```
class Validator:
    def is_valid(self, instance: object) -> TypeIs[str]:
        return isinstance(instance, str)

    def process(self, to_validate: object) -> None:
        if Validator().is_valid(to_validate):
            reveal_type(to_validate) # Revealed type is 'str'
            print(to_validate.upper()) # ok: to_validate is str
```

Assignment Expressions with Typels

You can use the assignment expression operator `:=` with `TypeIs` to create a new variable and narrow its type simultaneously.

```
from typing import TypeIs, reveal_type

def is_float(x: object) -> TypeIs[float]:
    return isinstance(x, float)

def main(a: object) -> None:
    if is_float(x := a):
        reveal_type(x) # Revealed type is 'float'
        # x is narrowed to float in this block
        print(x + 1.0)
```

1.11.5 Limitations

Mypy's analysis is limited to individual symbols and it will not track relationships between symbols. For example, in the following code it's easy to deduce that if `a` is `None` then `b` must not be, therefore `a or b` will always be an instance of `C`, but Mypy will not be able to tell that:

```
class C:
    pass

def f(a: C | None, b: C | None) -> C:
    if a is not None or b is not None:
        return a or b # Incompatible return value type (got "C | None", expected "C")
    return C()
```

Tracking these sort of cross-variable conditions in a type checker would add significant complexity and performance overhead.

You can use an `assert` to convince the type checker, override it with a *cast* or rewrite the function to be slightly more verbose:

```
def f(a: C | None, b: C | None) -> C:
    if a is not None:
        return a
    elif b is not None:
        return b
    return C()
```

1.12 Duck type compatibility

In Python, certain types are compatible even though they aren't subclasses of each other. For example, `int` objects are valid whenever `float` objects are expected. Mypy supports this idiom via *duck type compatibility*. This is supported for a small set of built-in types:

- `int` is duck type compatible with `float` and `complex`.
- `float` is duck type compatible with `complex`.

Note

`bytearray` and `memoryview` were duck type compatible with `bytes` by default prior to mypy 2.0. This can still be enabled with `--no-strict-bytes`.

For example, mypy considers an `int` object to be valid whenever a `float` object is expected. Thus code like this is nice and clean and also behaves as expected:

```
import math

def degrees_to_radians(degrees: float) -> float:
    return math.pi * degrees / 180

n = 90 # Inferred type 'int'
print(degrees_to_radians(n)) # Okay!
```

You can also often use *Protocols and structural subtyping* to achieve a similar effect in a more principled and extensible fashion. Protocols don't apply to cases like `int` being compatible with `float`, since `float` is not a protocol class but a regular, concrete class, and many standard library functions expect concrete instances of `float` (or `int`).

1.13 Stub files

A *stub file* is a file containing a skeleton of the public interface of that Python module, including classes, variables, functions – and most importantly, their types.

Mypy uses stub files stored in the `typeshed` repository to determine the types of standard library and third-party library functions, classes, and other definitions. You can also create your own stubs that will be used to type check your code.

1.13.1 Creating a stub

Here is an overview of how to create a stub file:

- Write a stub file for the library (or an arbitrary module) and store it as a `.pyi` file in the same directory as the library module.
- Alternatively, put your stubs (`.pyi` files) in a directory reserved for stubs (e.g., `myproject/stubs`). In this case you have to set the environment variable `MYPYPATH` to refer to the directory. For example:

```
$ export MYPYPATH=~/.work/myproject/stubs
```

Use the normal Python file name conventions for modules, e.g. `csv.pyi` for module `csv`. Use a subdirectory with `__init__.pyi` for packages. Note that [PEP 561](#) stub-only packages must be installed, and may not be pointed at through the `MYPYPATH` (see [PEP 561 support](#)).

If a directory contains both a `.py` and a `.pyi` file for the same module, the `.pyi` file takes precedence. This way you can easily add annotations for a module even if you don't want to modify the source code. This can be useful, for example, if you use 3rd party open source libraries in your program (and there are no stubs in `typeshed` yet).

That's it!

Now you can access the module in mypy programs and type check code that uses the library. If you write a stub for a library module, consider making it available for other programmers that use mypy by contributing it back to the `typeshed` repo.

Mypy also ships with two tools for making it easier to create and maintain stubs: *Automatic stub generation* (`stubgen`) and *Automatic stub testing* (`stubtest`).

The following sections explain the kinds of type annotations you can use in your programs and stub files.

Note

You may be tempted to point MYPYPATH to the standard library or to the `site-packages` directory where your 3rd party packages are installed. This is almost always a bad idea – you will likely get tons of error messages about code you didn’t write and that mypy can’t analyze all that well yet, and in the worst case scenario mypy may crash due to some construct in a 3rd party package that it didn’t expect.

1.13.2 Stub file syntax

Stub files are written in normal Python syntax, but generally leaving out runtime logic like variable initializers, function bodies, and default arguments.

If it is not possible to completely leave out some piece of runtime logic, the recommended convention is to replace or elide them with ellipsis expressions (`...`). Each ellipsis below is literally written in the stub file as three dots:

```
# Variables with annotations do not need to be assigned a value.
# So by convention, we omit them in the stub file.
x: int

# Function bodies cannot be completely removed. By convention,
# we replace them with `...` instead of the `pass` statement.
def func_1(code: str) -> int: ...

# We can do the same with default arguments.
def func_2(a: int, b: int = ...) -> int: ...
```

Note

The ellipsis `...` is also used with a different meaning in *callable types* and *tuple types*.

1.13.3 Using stub file syntax at runtime

You may also occasionally need to elide actual logic in regular Python code – for example, when writing methods in *overload variants* or *custom protocols*.

The recommended style is to use ellipses to do so, just like in stub files. It is also considered stylistically acceptable to throw a `NotImplementedError` in cases where the user of the code may accidentally call functions with no actual logic.

You can also elide default arguments as long as the function body also contains no runtime logic: the function body only contains a single ellipsis, the pass statement, or a `raise NotImplementedError()`. It is also acceptable for the function body to contain a docstring. For example:

```
from typing import Protocol

class Resource(Protocol):
    def ok_1(self, foo: list[str] = ...) -> None: ...

    def ok_2(self, foo: list[str] = ...) -> None:
        raise NotImplementedError()
```

(continues on next page)

(continued from previous page)

```
def ok_3(self, foo: list[str] = ...) -> None:
    """Some docstring"""
    pass

# Error: Incompatible default for parameter "foo" (default has
# type "ellipsis", parameter has type "list[str]")
def not_ok(self, foo: list[str] = ...) -> None:
    print(foo)
```

1.14 Generics

This section explains how you can define your own generic classes that take one or more type arguments, similar to built-in types such as `list[T]`. User-defined generics are a moderately advanced feature and you can get far without ever using them – feel free to skip this section and come back later.

1.14.1 Defining generic classes

The built-in collection classes are generic classes. Generic types accept one or more type arguments within `[...]`, which can be arbitrary types. For example, the type `dict[int, str]` has the type arguments `int` and `str`, and `list[int]` has the type argument `int`.

Programs can also define new generic classes. Here is a very simple generic class that represents a stack (using the syntax introduced in Python 3.12):

```
class Stack[T]:
    def __init__(self) -> None:
        # Create an empty list with items of type T
        self.items: list[T] = []

    def push(self, item: T) -> None:
        self.items.append(item)

    def pop(self) -> T:
        return self.items.pop()

    def empty(self) -> bool:
        return not self.items
```

There are two syntax variants for defining generic classes in Python. Python 3.12 introduced a [new dedicated syntax](#) for defining generic classes (and also functions and type aliases, which we will discuss later). The above example used the new syntax. Most examples are given using both the new and the old (or legacy) syntax variants. Unless mentioned otherwise, they work the same – but the new syntax is more readable and more convenient.

Here is the same example using the old syntax (required for Python 3.11 and earlier, but also supported on newer Python versions):

```
from typing import TypeVar, Generic

T = TypeVar('T') # Define type variable "T"

class Stack(Generic[T]):
    def __init__(self) -> None:
```

(continues on next page)

(continued from previous page)

```

    # Create an empty list with items of type T
    self.items: list[T] = []

    def push(self, item: T) -> None:
        self.items.append(item)

    def pop(self) -> T:
        return self.items.pop()

    def empty(self) -> bool:
        return not self.items

```

Note

There are currently no plans to deprecate the legacy syntax. You can freely mix code using the new and old syntax variants, even within a single file (but *not* within a single class).

The `Stack` class can be used to represent a stack of any type: `Stack[int]`, `Stack[tuple[int, str]]`, etc. You can think of `Stack[int]` as referring to the definition of `Stack` above, but with all instances of `T` replaced with `int`.

Using `Stack` is similar to built-in container types:

```

# Construct an empty Stack[int] instance
stack = Stack[int]()
stack.push(2)
stack.pop()

# error: Argument 1 to "push" of "Stack" has incompatible type "str"; expected "int"
stack.push('x')

stack2: Stack[str] = Stack()
stack2.push('x')

```

Construction of instances of generic types is type checked (Python 3.12 syntax):

```

class Box[T]:
    def __init__(self, content: T) -> None:
        self.content = content

Box(1)           # OK, inferred type is Box[int]
Box[int](1)      # Also OK

# error: Argument 1 to "Box" has incompatible type "str"; expected "int"
Box[int]('some string')

```

Here is the definition of `Box` using the legacy syntax (Python 3.11 and earlier):

```

from typing import TypeVar, Generic

T = TypeVar('T')

class Box(Generic[T]):

```

(continues on next page)

(continued from previous page)

```
def __init__(self, content: T) -> None:
    self.content = content
```

Note

Before moving on, let's clarify some terminology. The name `T` in `class Stack[T]` or `class Stack(Generic[T])` declares a *type parameter* `T` (of class `Stack`). `T` is also called a *type variable*, especially in a type annotation, such as in the signature of `push` above. When the type `Stack[...]` is used in a type annotation, the type within square brackets is called a *type argument*. This is similar to the distinction between function parameters and arguments.

1.14.2 Defining subclasses of generic classes

User-defined generic classes and generic classes defined in `typing` can be used as a base class for another class (generic or non-generic). For example (Python 3.12 syntax):

```
from typing import Mapping, Iterator

# This is a generic subclass of Mapping
class MyMap[KT, VT](Mapping[KT, VT]):
    def __getitem__(self, k: KT) -> VT: ...
    def __iter__(self) -> Iterator[KT]: ...
    def __len__(self) -> int: ...

items: MyMap[str, int] # OK

# This is a non-generic subclass of dict
class StrDict(dict[str, str]):
    def __str__(self) -> str:
        return f'StrDict({super().__str__()})'

data: StrDict[int, int] # Error! StrDict is not generic
data2: StrDict # OK

# This is a user-defined generic class
class Receiver[T]:
    def accept(self, value: T) -> None: ...

# This is a generic subclass of Receiver
class AdvancedReceiver[T](Receiver[T]): ...
```

Here is the above example using the legacy syntax (Python 3.11 and earlier):

```
from typing import Generic, TypeVar, Mapping, Iterator

KT = TypeVar('KT')
VT = TypeVar('VT')

# This is a generic subclass of Mapping
class MyMap(Generic[KT, VT]):
    def __getitem__(self, k: KT) -> VT: ...
```

(continues on next page)

(continued from previous page)

```

def __iter__(self) -> Iterator[KT]: ...
def __len__(self) -> int: ...

items: MyMap[str, int] # OK

# This is a non-generic subclass of dict
class StrDict(dict[str, str]):
    def __str__(self) -> str:
        return f'StrDict({super().__str__()})'

data: StrDict[int, int] # Error! StrDict is not generic
data2: StrDict # OK

# This is a user-defined generic class
class Receiver(Generic[T]):
    def accept(self, value: T) -> None: ...

# This is a generic subclass of Receiver
class AdvancedReceiver(Receiver[T]): ...

```

Note

You have to add an explicit `Mapping` base class if you want mypy to consider a user-defined class as a mapping (and `Sequence` for sequences, etc.). This is because mypy doesn't use *structural subtyping* for these ABCs, unlike simpler protocols like `Iterable`, which use *structural subtyping*.

When using the legacy syntax, `Generic` can be omitted from bases if there are other base classes that include type variables, such as `Mapping[KT, VT]` in the above example. If you include `Generic[...]` in bases, then it should list all type variables present in other bases (or more, if needed). The order of type parameters is defined by the following rules:

- If `Generic[...]` is present, then the order of parameters is always determined by their order in `Generic[...]`.
- If there are no `Generic[...]` in bases, then all type parameters are collected in the lexicographic order (i.e. by first appearance).

Example:

```

from typing import Generic, TypeVar, Any

T = TypeVar('T')
S = TypeVar('S')
U = TypeVar('U')

class One(Generic[T]): ...
class Another(Generic[T]): ...

class First(One[T], Another[S]): ...
class Second(One[T], Another[S], Generic[S, U, T]): ...

x: First[int, str] # Here T is bound to int, S is bound to str
y: Second[int, str, Any] # Here T is Any, S is int, and U is str

```

When using the Python 3.12 syntax, all type parameters must always be explicitly defined immediately after the class name within [...], and the `Generic[...]` base class is never used.

1.14.3 Generic functions

Functions can also be generic, i.e. they can have type parameters (Python 3.12 syntax):

```
from collections.abc import Sequence

# A generic function!
def first[T](seq: Sequence[T]) -> T:
    return seq[0]
```

Here is the same example using the legacy syntax (Python 3.11 and earlier):

```
from typing import TypeVar, Sequence

T = TypeVar('T')

# A generic function!
def first(seq: Sequence[T]) -> T:
    return seq[0]
```

As with generic classes, the type parameter `T` can be replaced with any type. That means `first` can be passed an argument with any sequence type, and the return type is derived from the sequence item type. Example:

```
reveal_type(first([1, 2, 3]))    # Revealed type is "builtins.int"
reveal_type(first(('a', 'b')))  # Revealed type is "builtins.str"
```

When using the legacy syntax, a single definition of a type variable (such as `T` above) can be used in multiple generic functions or classes. In this example we use the same type variable in two generic functions to declare type parameters:

```
from typing import TypeVar, Sequence

T = TypeVar('T')    # Define type variable

def first(seq: Sequence[T]) -> T:
    return seq[0]

def last(seq: Sequence[T]) -> T:
    return seq[-1]
```

Since the Python 3.12 syntax is more concise, it doesn't need (or have) an equivalent way of sharing type parameter definitions.

A variable cannot have a type variable in its type unless the type variable is bound in a containing generic class or function.

When calling a generic function, you can't explicitly pass the values of type parameters as type arguments. The values of type parameters are always inferred by mypy. This is not valid:

```
first[int]([1, 2])  # Error: can't use [...] with generic function
```

If you really need this, you can define a generic class with a `__call__` method.

1.14.4 Type variables with upper bounds

A type variable can also be restricted to having values that are subtypes of a specific type. This type is called the upper bound of the type variable, and it is specified using `T: <bound>` when using the Python 3.12 syntax. In the definition of a generic function or a generic class that uses such a type variable `T`, the type represented by `T` is assumed to be a subtype of its upper bound, so you can use methods of the upper bound on values of type `T` (Python 3.12 syntax):

```
from typing import SupportsAbs

def max_by_abs[T: SupportsAbs[float]](*xs: T) -> T:
    # We can use abs(), because T is a subtype of SupportsAbs[float].
    return max(xs, key=abs)
```

An upper bound can also be specified with the `bound=...` keyword argument to `TypeVar`. Here is the example using the legacy syntax (Python 3.11 and earlier):

```
from typing import TypeVar, SupportsAbs

T = TypeVar('T', bound=SupportsAbs[float])

def max_by_abs(*xs: T) -> T:
    return max(xs, key=abs)
```

In a call to such a function, the type `T` must be replaced by a type that is a subtype of its upper bound. Continuing the example above:

```
max_by_abs(-3.5, 2)    # Okay, has type 'float'
max_by_abs(5+6j, 7)    # Okay, has type 'complex'
max_by_abs('a', 'b')  # Error: 'str' is not a subtype of SupportsAbs[float]
```

Type parameters of generic classes may also have upper bounds, which restrict the valid values for the type parameter in the same way.

1.14.5 Generic methods and generic self

You can also define generic methods. In particular, the `self` parameter may also be generic, allowing a method to return the most precise type known at the point of access. In this way, for example, you can type check a chain of setter methods (Python 3.12 syntax):

```
class Shape:
    def set_scale[T: Shape](self: T, scale: float) -> T:
        self.scale = scale
        return self

class Circle(Shape):
    def set_radius(self, r: float) -> 'Circle':
        self.radius = r
        return self

class Square(Shape):
    def set_width(self, w: float) -> 'Square':
        self.width = w
        return self
```

(continues on next page)

(continued from previous page)

```
circle: Circle = Circle().set_scale(0.5).set_radius(2.7)
square: Square = Square().set_scale(0.5).set_width(3.2)
```

Without using generic `self`, the last two lines could not be type checked properly, since the return type of `set_scale` would be `Shape`, which doesn't define `set_radius` or `set_width`.

When using the legacy syntax, just use a type variable in the method signature that is different from class type parameters (if any are defined). Here is the above example using the legacy syntax (3.11 and earlier):

```
from typing import TypeVar

T = TypeVar('T', bound='Shape')

class Shape:
    def set_scale(self: T, scale: float) -> T:
        self.scale = scale
        return self

class Circle(Shape):
    def set_radius(self, r: float) -> 'Circle':
        self.radius = r
        return self

class Square(Shape):
    def set_width(self, w: float) -> 'Square':
        self.width = w
        return self

circle: Circle = Circle().set_scale(0.5).set_radius(2.7)
square: Square = Square().set_scale(0.5).set_width(3.2)
```

Other uses include factory methods, such as copy and deserialization methods. For class methods, you can also define generic cls, using `type[T]` or `Type[T]` (Python 3.12 syntax):

```
class Friend:
    other: "Friend | None" = None

    @classmethod
    def make_pair[T: Friend](cls: type[T]) -> tuple[T, T]:
        a, b = cls(), cls()
        a.other = b
        b.other = a
        return a, b

class SuperFriend(Friend):
    pass

a, b = SuperFriend.make_pair()
```

Here is the same example using the legacy syntax (3.11 and earlier):

```
from typing import TypeVar
```

(continues on next page)

(continued from previous page)

```

T = TypeVar('T', bound='Friend')

class Friend:
    other: "Friend | None" = None

    @classmethod
    def make_pair(cls: type[T]) -> tuple[T, T]:
        a, b = cls(), cls()
        a.other = b
        b.other = a
        return a, b

class SuperFriend(Friend):
    pass

a, b = SuperFriend.make_pair()

```

Note that when overriding a method with generic `self`, you must either return a generic `self` too, or return an instance of the current class. In the latter case, you must implement this method in all future subclasses.

Note also that mypy cannot always verify that the implementation of a copy or a deserialization method returns the actual type of `self`. Therefore you may need to silence mypy inside these methods (but not at the call site), possibly by making use of the `Any` type or a `# type: ignore` comment.

Mypy lets you use generic `self` types in certain unsafe ways in order to support common idioms. For example, using a generic `self` type in an argument type is accepted even though it's unsafe (Python 3.12 syntax):

```

class Base:
    def compare[T: Base](self: T, other: T) -> bool:
        return False

class Sub(Base):
    def __init__(self, x: int) -> None:
        self.x = x

    # This is unsafe (see below) but allowed because it's
    # a common pattern and rarely causes issues in practice.
    def compare(self, other: 'Sub') -> bool:
        return self.x > other.x

b: Base = Sub(42)
b.compare(Base()) # Runtime error here: 'Base' object has no attribute 'x'

```

For some advanced uses of `self` types, see *additional examples*.

1.14.6 Automatic self types using `typing.Self`

Since the patterns described above are quite common, mypy supports a simpler syntax, introduced in [PEP 673](#), to make them easier to use. Instead of introducing a type parameter and using an explicit annotation for `self`, you can import the special type `typing.Self` that is automatically transformed into a method-level type parameter with the current class as the upper bound, and you don't need an annotation for `self` (or `cls` in class methods). The example from the previous section can be made simpler by using `Self`:

```

from typing import Self

class Friend:
    other: Self | None = None

    @classmethod
    def make_pair(cls) -> tuple[Self, Self]:
        a, b = cls(), cls()
        a.other = b
        b.other = a
        return a, b

class SuperFriend(Friend):
    pass

a, b = SuperFriend.make_pair()

```

This is more compact than using explicit type parameters. Also, you can use `Self` in attribute annotations in addition to methods.

Note

To use this feature on Python versions earlier than 3.11, you will need to import `Self` from `typing_extensions` (version 4.0 or newer).

1.14.7 Variance of generic types

There are three main kinds of generic types with respect to subtype relations between them: invariant, covariant, and contravariant. Assuming that we have a pair of types `A` and `B`, and `B` is a subtype of `A`, these are defined as follows:

- A generic class `MyCovGen[T]` is called covariant in type variable `T` if `MyCovGen[B]` is always a subtype of `MyCovGen[A]`.
- A generic class `MyContraGen[T]` is called contravariant in type variable `T` if `MyContraGen[A]` is always a subtype of `MyContraGen[B]`.
- A generic class `MyInvGen[T]` is called invariant in `T` if neither of the above is true.

Let us illustrate this by few simple examples:

```

# We'll use these classes in the examples below
class Shape: ...
class Triangle(Shape): ...
class Square(Shape): ...

```

- Most immutable container types, such as `Sequence` and `frozenset` are covariant. Union types are also covariant in all union items: `Triangle | int` is a subtype of `Shape | int`.

```

def count_lines(shapes: Sequence[Shape]) -> int:
    return sum(shape.num_sides for shape in shapes)

triangles: Sequence[Triangle]
count_lines(triangles) # OK

```

(continues on next page)

(continued from previous page)

```
def foo(triangle: Triangle, num: int) -> None:
    shape_or_number: Union[Shape, int]
    # a Triangle is a Shape, and a Shape is a valid Union[Shape, int]
    shape_or_number = triangle
```

Covariance should feel relatively intuitive, but contravariance and invariance can be harder to reason about.

- `Callable` is an example of type that behaves contravariant in types of arguments. That is, `Callable[[Shape], int]` is a subtype of `Callable[[Triangle], int]`, despite `Shape` being a supertype of `Triangle`. To understand this, consider:

```
def cost_of_paint_required(
    triangle: Triangle,
    area_calculator: Callable[[Triangle], float]
) -> float:
    return area_calculator(triangle) * DOLLAR_PER_SQ_FT

# This straightforwardly works
def area_of_triangle(triangle: Triangle) -> float: ...
cost_of_paint_required(triangle, area_of_triangle) # OK

# But this works as well!
def area_of_any_shape(shape: Shape) -> float: ...
cost_of_paint_required(triangle, area_of_any_shape) # OK
```

`cost_of_paint_required` needs a callable that can calculate the area of a triangle. If we give it a callable that can calculate the area of an arbitrary shape (not just triangles), everything still works.

- `list` is an invariant generic type. Naively, one would think that it is covariant, like `Sequence` above, but consider this code:

```
class Circle(Shape):
    # The rotate method is only defined on Circle, not on Shape
    def rotate(self): ...

def add_one(things: list[Shape]) -> None:
    things.append(Shape())

my_circles: list[Circle] = []
add_one(my_circles) # This may appear safe, but...
my_circles[0].rotate() # ...this will fail, since my_circles[0] is now a Shape,
                        ↪ not a Circle
```

Another example of invariant type is `dict`. Most mutable containers are invariant.

When using the Python 3.12 syntax for generics, mypy will automatically infer the most flexible variance for each class type variable. Here `Box` will be inferred as covariant:

```
class Box[T]: # this type is implicitly covariant
    def __init__(self, content: T) -> None:
        self._content = content

    def get_content(self) -> T:
        return self._content
```

(continues on next page)

(continued from previous page)

```
def look_into(box: Box[Shape]): ...

my_box = Box(Square())
look_into(my_box)  # OK, but mypy would complain here for an invariant type
```

Here the underscore prefix for `_content` is significant. Without an underscore prefix, the class would be invariant, as the attribute would be understood as a public, mutable attribute (a single underscore prefix has no special significance for mypy in most other contexts). By declaring the attribute as `Final`, the class could still be made covariant:

```
from typing import Final

class Box[T]:  # this type is implicitly covariant
    def __init__(self, content: T) -> None:
        self.content: Final = content

    def get_content(self) -> T:
        return self.content
```

When using the legacy syntax, mypy assumes that all user-defined generics are invariant by default. To declare a given generic class as covariant or contravariant, use type variables defined with special keyword arguments `covariant` or `contravariant`. For example (Python 3.11 or earlier):

```
from typing import Generic, TypeVar

T_co = TypeVar('T_co', covariant=True)

class Box(Generic[T_co]):  # this type is declared covariant
    def __init__(self, content: T_co) -> None:
        self._content = content

    def get_content(self) -> T_co:
        return self._content

def look_into(box: Box[Shape]): ...

my_box = Box(Square())
look_into(my_box)  # OK, but mypy would complain here for an invariant type
```

1.14.8 Value-constrained type variables

By default, a type variable can be replaced with any type – or any type that is a subtype of the upper bound, which defaults to `object`. However, sometimes it's useful to have a type variable that can only have some specific types as its value. A typical example is a type variable that can only have values `str` and `bytes`. This lets us define a function that can concatenate two strings or bytes objects, but it can't be called with other argument types (Python 3.12 syntax):

```
def concat[S: (str, bytes)](x: S, y: S) -> S:
    return x + y

concat('a', 'b')  # Okay
concat(b'a', b'b')  # Okay
concat(1, 2)      # Error!
```


The same thing is also possibly using the legacy syntax (Python 3.11 or earlier):

```
from typing import TypeVar

AnyStr = TypeVar('AnyStr', str, bytes)

def concat(x: AnyStr, y: AnyStr) -> AnyStr:
    return x + y
```

No matter which syntax you use, such a type variable is called a value-constrained type variable (the allowed types are accessible at runtime via `TypeVar.__constraints__`). Importantly, this is different from a union type, since combinations of `str` and `bytes` are not accepted:

```
concat('string', b'bytes') # Error!
```

In this case, this is exactly what we want, since it's not possible to concatenate a string and a bytes object! If we tried to use a union type, the type checker would complain about this possibility:

```
def union_concat(x: str | bytes, y: str | bytes) -> str | bytes:
    return x + y # Error: can't concatenate str and bytes
```

Another interesting special case is calling `concat()` with a subtype of `str`:

```
class S(str): pass

ss = concat(S('foo'), S('bar'))
reveal_type(ss) # Revealed type is "builtins.str"
```

You may expect that the type of `ss` is `S`, but the type is actually `str`: a subtype gets promoted to one of the valid values for the type variable, which in this case is `str`.

This is thus subtly different from using `str | bytes` as an upper bound, where the return type would be `S` (see *Type variables with upper bounds*). Using a value constraint is correct for `concat`, since `concat` actually returns a `str` instance in the above example:

```
>>> print(type(ss))
<class 'str'>
```

You can also use type variables with a restricted set of possible values when defining a generic class. For example, the type `Pattern[S]` is used for the return value of `re.compile()`, where `S` can be either `str` or `bytes`. Regular expressions can be based on a string or a bytes pattern.

A type variable may not have both value constraints and an upper bound.

Note that you may come across `AnyStr` imported from `typing`. This feature is now deprecated, but it means the same as our definition of `AnyStr` above.

1.14.9 Declaring decorators

Decorators are typically functions that take a function as an argument and return another function. Describing this behaviour in terms of types can be a little tricky; we'll show how you can use type variables and a special kind of type variable called a *parameter specification* to do so.

Suppose we have the following decorator, not type annotated yet, that preserves the original function's signature and merely prints the decorated function's name:

```
def printing_decorator(func):
    def wrapper(*args, **kwds):
        print("Calling", func)
        return func(*args, **kwds)
    return wrapper
```

We can use it to decorate function `add_forty_two`:

```
# A decorated function.
@printing_decorator
def add_forty_two(value: int) -> int:
    return value + 42

a = add_forty_two(3)
```

Since `printing_decorator` is not type-annotated, the following won't get type checked:

```
reveal_type(a)           # Revealed type is "Any"
add_forty_two('foo')     # No type checker error :(
```

This is a sorry state of affairs! If you run with `--strict`, mypy will even alert you to this fact: `Untyped decorator makes function "add_forty_two" untyped`

Note that class decorators are handled differently than function decorators in mypy: decorating a class does not erase its type, even if the decorator has incomplete type annotations.

Here's how one could annotate the decorator (Python 3.12 syntax):

```
from collections.abc import Callable
from typing import Any, cast

# A decorator that preserves the signature.
def printing_decorator[F: Callable[..., Any]](func: F) -> F:
    def wrapper(*args, **kwds):
        print("Calling", func)
        return func(*args, **kwds)
    return cast(F, wrapper)

@printing_decorator
def add_forty_two(value: int) -> int:
    return value + 42

a = add_forty_two(3)
reveal_type(a)           # Revealed type is "builtins.int"
add_forty_two('x')       # Argument 1 to "add_forty_two" has incompatible type "str";
                           ↪ expected "int"
```

Here is the example using the legacy syntax (Python 3.11 and earlier):

```
from collections.abc import Callable
from typing import Any, TypeVar, cast

F = TypeVar('F', bound=Callable[..., Any])
```

(continues on next page)

(continued from previous page)

```
# A decorator that preserves the signature.
def printing_decorator(func: F) -> F:
    def wrapper(*args, **kwds):
        print("Calling", func)
        return func(*args, **kwds)
    return cast(F, wrapper)

@printing_decorator
def add_forty_two(value: int) -> int:
    return value + 42

a = add_forty_two(3)
reveal_type(a)      # Revealed type is "builtins.int"
add_forty_two('x')  # Argument 1 to "add_forty_two" has incompatible type "str";
                    # ↪ expected "int"
```

This still has some shortcomings. First, we need to use the unsafe `cast()` to convince mypy that `wrapper()` has the same signature as `func` (see *casts*).

Second, the `wrapper()` function is not tightly type checked, although wrapper functions are typically small enough that this is not a big problem. This is also the reason for the `cast()` call in the `return` statement in `printing_decorator()`.

However, we can use a parameter specification, introduced using `**P`, for a more faithful type annotation (Python 3.12 syntax):

```
from collections.abc import Callable

def printing_decorator[*P, T](func: Callable[P, T]) -> Callable[P, T]:
    def wrapper(*args: P.args, **kwds: P.kwargs) -> T:
        print("Calling", func)
        return func(*args, **kwds)
    return wrapper
```

The same is possible using the legacy syntax with `ParamSpec` (Python 3.11 and earlier):

```
from collections.abc import Callable
from typing import TypeVar
from typing_extensions import ParamSpec

P = ParamSpec('P')
T = TypeVar('T')

def printing_decorator(func: Callable[P, T]) -> Callable[P, T]:
    def wrapper(*args: P.args, **kwds: P.kwargs) -> T:
        print("Calling", func)
        return func(*args, **kwds)
    return wrapper
```

Parameter specifications also allow you to describe decorators that alter the signature of the input function (Python 3.12 syntax):

```
from collections.abc import Callable
```

(continues on next page)

(continued from previous page)

```
# We reuse 'P' in the return type, but replace 'T' with 'str'
def stringify[**P, T](func: Callable[P, T]) -> Callable[P, str]:
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> str:
        return str(func(*args, **kwargs))
    return wrapper

@stringify
def add_forty_two(value: int) -> int:
    return value + 42

a = add_forty_two(3)
reveal_type(a)      # Revealed type is "builtins.str"
add_forty_two('x')  # error: Argument 1 to "add_forty_two" has incompatible type "str";
                    # expected "int"
```

Here is the above example using the legacy syntax (Python 3.11 and earlier):

```
from collections.abc import Callable
from typing import TypeVar
from typing_extensions import ParamSpec

P = ParamSpec('P')
T = TypeVar('T')

# We reuse 'P' in the return type, but replace 'T' with 'str'
def stringify(func: Callable[P, T]) -> Callable[P, str]:
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> str:
        return str(func(*args, **kwargs))
    return wrapper
```

You can also insert an argument in a decorator (Python 3.12 syntax):

```
from collections.abc import Callable
from typing import Concatenate

def printing_decorator[**P, T](func: Callable[P, T]) -> Callable[Concatenate[str, P], T]:
    def wrapper(msg: str, /, *args: P.args, **kwargs: P.kwargs) -> T:
        print("Calling", func, "with", msg)
        return func(*args, **kwargs)
    return wrapper

@printing_decorator
def add_forty_two(value: int) -> int:
    return value + 42

a = add_forty_two('three', 3)
```

Here is the same function using the legacy syntax (Python 3.11 and earlier):

```
from collections.abc import Callable
from typing import TypeVar
from typing_extensions import Concatenate, ParamSpec
```

(continues on next page)

(continued from previous page)

```

P = ParamSpec('P')
T = TypeVar('T')

def printing_decorator(func: Callable[P, T]) -> Callable[Concatenate[str, P], T]:
    def wrapper(msg: str, /, *args: P.args, **kwargs: P.kwargs) -> T:
        print("Calling", func, "with", msg)
        return func(*args, **kwargs)
    return wrapper

```

Decorator factories

Functions that take arguments and return a decorator (also called second-order decorators), are similarly supported via generics (Python 3.12 syntax):

```

from collections.abc import Callable
from typing import Any

def route[F: Callable[..., Any]](url: str) -> Callable[[F], F]:
    ...

@route(url='/')
def index(request: Any) -> str:
    return 'Hello world'

```

Note that mypy infers that `F` is used to make the `Callable` return value of `route` generic, instead of making `route` itself generic, since `F` is only used in the return type. Python has no explicit syntax to mark that `F` is only bound in the return value.

Here is the example using the legacy syntax (Python 3.11 and earlier):

```

from collections.abc import Callable
from typing import Any, TypeVar

F = TypeVar('F', bound=Callable[..., Any])

def route(url: str) -> Callable[[F], F]:
    ...

@route(url='/')
def index(request: Any) -> str:
    return 'Hello world'

```

Sometimes the same decorator supports both bare calls and calls with arguments. This can be achieved by combining with `@overload` (Python 3.12 syntax):

```

from collections.abc import Callable
from typing import Any, overload

# Bare decorator usage
@overload
def atomic[F: Callable[..., Any]](func: F, /) -> F: ...
# Decorator with arguments
@overload

```

(continues on next page)

(continued from previous page)

```
def atomic[F: Callable[..., Any]](*, savepoint: bool = True) -> Callable[[F], F]: ...

# Implementation
def atomic(func: Callable[..., Any] | None = None, /, *, savepoint: bool = True):
    def decorator(func: Callable[..., Any]):
        ... # Code goes here
    if __func is not None:
        return decorator(__func)
    else:
        return decorator

# Usage
@atomic
def func1() -> None: ...

@atomic(savepoint=False)
def func2() -> None: ...
```

Here is the decorator from the example using the legacy syntax (Python 3.11 and earlier):

```
from collections.abc import Callable
from typing import Any, Optional, TypeVar, overload

F = TypeVar('F', bound=Callable[..., Any])

# Bare decorator usage
@overload
def atomic(func: F, /) -> F: ...
# Decorator with arguments
@overload
def atomic(*, savepoint: bool = True) -> Callable[[F], F]: ...

# Implementation
def atomic(func: Optional[Callable[..., Any]] = None, /, *, savepoint: bool = True):
    ... # Same as above
```

1.14.10 Generic protocols

Mypy supports generic protocols (see also *Protocols and structural subtyping*). Several *predefined protocols* are generic, such as `Iterable[T]`, and you can define additional generic protocols. Generic protocols mostly follow the normal rules for generic classes. Example (Python 3.12 syntax):

```
from typing import Protocol

class Box[T](Protocol):
    content: T

def do_stuff(one: Box[str], other: Box[bytes]) -> None:
    ...

class StringWrapper:
    def __init__(self, content: str) -> None:
```

(continues on next page)

(continued from previous page)

```

        self.content = content

class BytesWrapper:
    def __init__(self, content: bytes) -> None:
        self.content = content

do_stuff(StringWrapper('one'), BytesWrapper(b'other')) # OK

x: Box[float] = ...
y: Box[int] = ...
x = y # Error -- Box is invariant

```

Here is the definition of Box from the above example using the legacy syntax (Python 3.11 and earlier):

```

from typing import Protocol, TypeVar

T = TypeVar('T')

class Box(Protocol[T]):
    content: T

```

Note that `class ClassName(Protocol[T])` is allowed as a shorthand for `class ClassName(Protocol, Generic[T])` when using the legacy syntax, as per [PEP 544: Generic protocols](#). This form is only valid when using the legacy syntax.

When using the legacy syntax, there is an important difference between generic protocols and ordinary generic classes: mypy checks that the declared variances of generic type variables in a protocol match how they are used in the protocol definition. The protocol in this example is rejected, since the type variable `T` is used covariantly as a return type, but the type variable is invariant:

```

from typing import Protocol, TypeVar

T = TypeVar('T')

class ReadOnlyBox(Protocol[T]): # error: Invariant type variable "T" used in protocol,
    ↪ where covariant one is expected
    def content(self) -> T: ...

```

This example correctly uses a covariant type variable:

```

from typing import Protocol, TypeVar

T_co = TypeVar('T_co', covariant=True)

class ReadOnlyBox(Protocol[T_co]): # OK
    def content(self) -> T_co: ...

ax: ReadOnlyBox[float] = ...
ay: ReadOnlyBox[int] = ...
ax = ay # OK -- ReadOnlyBox is covariant

```

See *Variance of generic types* for more about variance.

Generic protocols can also be recursive. Example (Python 3.12 syntax):

```

class Linked[T](Protocol):
    val: T
    def next(self) -> 'Linked[T]': ...

class L:
    val: int
    def next(self) -> 'L': ...

def last(seq: Linked[T]) -> T: ...

result = last(L())
reveal_type(result)  # Revealed type is "builtins.int"

```

Here is the definition of `Linked` using the legacy syntax (Python 3.11 and earlier):

```

from typing import TypeVar

T = TypeVar('T')

class Linked(Protocol[T]):
    val: T
    def next(self) -> 'Linked[T]': ...

```

1.14.11 Generic type aliases

Type aliases can be generic. In this case they can be used in two ways. First, subscripted aliases are equivalent to original types with substituted type variables. Second, unsubscripted aliases are treated as original types with type parameters replaced with `Any`.

The `type` statement introduced in Python 3.12 is used to define generic type aliases (it also supports non-generic type aliases):

```

from collections.abc import Callable, Iterable

type TInt[S] = tuple[int, S]
type UInt[S] = S | int
type CBack[S] = Callable[..., S]

def response(query: str) -> UInt[str]:  # Same as str | int
    ...
def activate[S](cb: CBack[S]) -> S:      # Same as Callable[..., S]
    ...
table_entry: TInt  # Same as tuple[int, Any]

type Vec[T: (int, float, complex)] = Iterable[tuple[T, T]]

def inproduct[T: (int, float, complex)](v: Vec[T]) -> T:
    return sum(x*y for x, y in v)

def dilate[T: (int, float, complex)](v: Vec[T], scale: T) -> Vec[T]:
    return ((x * scale, y * scale) for x, y in v)

v1: Vec[int] = []  # Same as Iterable[tuple[int, int]]

```

(continues on next page)

(continued from previous page)

```
v2: Vec = []           # Same as Iterable[tuple[Any, Any]]
v3: Vec[int, int] = [] # Error: Invalid alias, too many type arguments!
```

There is also a legacy syntax that relies on `TypeVar`. Here the number of type arguments must match the number of free type variables in the generic type alias definition. A type variable is free if it's not a type parameter of a surrounding class or function. Example (following [PEP 484: Type aliases](#), Python 3.11 and earlier):

```
from typing import TypeVar, Iterable, Union, Callable

S = TypeVar('S')

TInt = tuple[int, S] # 1 type parameter, since only S is free
UInt = Union[S, int]
CBack = Callable[..., S]

def response(query: str) -> UInt[str]: # Same as Union[str, int]
    ...
def activate(cb: CBack[S]) -> S:      # Same as Callable[..., S]
    ...
table_entry: TInt # Same as tuple[int, Any]

T = TypeVar('T', int, float, complex)

Vec = Iterable[tuple[T, T]]

def inproduct(v: Vec[T]) -> T:
    return sum(x*y for x, y in v)

def dilate(v: Vec[T], scale: T) -> Vec[T]:
    return ((x * scale, y * scale) for x, y in v)

v1: Vec[int] = []           # Same as Iterable[tuple[int, int]]
v2: Vec = []               # Same as Iterable[tuple[Any, Any]]
v3: Vec[int, int] = []     # Error: Invalid alias, too many type arguments!
```

Type aliases can be imported from modules just like other names. An alias can also target another alias, although building complex chains of aliases is not recommended – this impedes code readability, thus defeating the purpose of using aliases. Example (Python 3.12 syntax):

```
from example1 import AliasType
from example2 import Vec

# AliasType and Vec are type aliases (Vec as defined above)

def fun() -> AliasType:
    ...

type OIntVec = Vec[int] | None
```

Type aliases defined using the `type` statement are not valid as base classes, and they can't be used to construct instances:

```
from example1 import AliasType
from example2 import Vec
```

(continues on next page)

(continued from previous page)

```
# AliasType and Vec are type aliases (Vec as defined above)

class NewVec[T](Vec[T]): # Error: not valid as base class
    ...

x = AliasType() # Error: can't be used to create instances
```

Here are examples using the legacy syntax (Python 3.11 and earlier):

```
from typing import TypeVar, Generic, Optional
from example1 import AliasType
from example2 import Vec

# AliasType and Vec are type aliases (Vec as defined above)

def fun() -> AliasType:
    ...

OIntVec = Optional[Vec[int]]

T = TypeVar('T')

# Old-style type aliases can be used as base classes and you can
# construct instances using them

class NewVec(Vec[T]):
    ...

x = AliasType()

for i, j in NewVec[int]():
    ...
```

Using type variable bounds or value constraints in generic aliases has the same effect as in generic classes and functions.

1.14.12 Differences between the new and old syntax

There are a few notable differences between the new (Python 3.12 and later) and the old syntax for generic classes, functions and type aliases, beyond the obvious syntactic differences:

- Type variables defined using the old syntax create definitions at runtime in the surrounding namespace, whereas the type variables defined using the new syntax are only defined within the class, function or type variable that uses them.
- Type variable definitions can be shared when using the old syntax, but the new syntax doesn't support this.
- When using the new syntax, the variance of class type variables is always inferred.
- Type aliases defined using the new syntax can contain forward references and recursive references without using string literal escaping. The same is true for the bounds and constraints of type variables.
- The new syntax lets you define a generic alias where the definition doesn't contain a reference to a type parameter. This is occasionally useful, at least when conditionally defining type aliases.

- Type aliases defined using the new syntax can't be used as base classes and can't be used to construct instances, unlike aliases defined using the old syntax.

1.14.13 Generic class internals

You may wonder what happens at runtime when you index a generic class. Indexing returns a *generic alias* to the original class that returns instances of the original class on instantiation (Python 3.12 syntax):

```
>>> class Stack[T]: ...
>>> Stack
__main__.Stack
>>> Stack[int]
__main__.Stack[int]
>>> instance = Stack[int]()
>>> instance.__class__
__main__.Stack
```

Here is the same example using the legacy syntax (Python 3.11 and earlier):

```
>>> from typing import TypeVar, Generic
>>> T = TypeVar('T')
>>> class Stack(Generic[T]): ...
>>> Stack
__main__.Stack
>>> Stack[int]
__main__.Stack[int]
>>> instance = Stack[int]()
>>> instance.__class__
__main__.Stack
```

Generic aliases can be instantiated or subclassed, similar to real classes, but the above examples illustrate that type variables are erased at runtime. Generic `Stack` instances are just ordinary Python objects, and they have no extra runtime overhead or magic due to being generic, other than the `Generic` base class that overloads the indexing operator using `__class_getitem__`. `typing.Generic` is included as an implicit base class even when using the new syntax:

```
>>> class Stack[T]: ...
>>> Stack.mro()
[<class '__main__.Stack'>, <class 'typing.Generic'>, <class 'object'>]
```

Note that in Python 3.8 and earlier, the built-in types `list`, `dict` and others do not support indexing. This is why we have the aliases `List`, `Dict` and so on in the `typing` module. Indexing these aliases gives you a generic alias that resembles generic aliases constructed by directly indexing the target class in more recent versions of Python:

```
>>> # Prefer the 'list[int]' syntax
>>> from typing import List
>>> List[int]
typing.List[int]
```

Note that the generic aliases in `typing` don't support constructing instances, unlike the corresponding built-in classes:

```
>>> list[int]()
[]
>>> from typing import List
>>> List[int]()
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
TypeError: Type List cannot be instantiated; use list() instead
```

1.15 More types

This section introduces a few additional kinds of types, including `NoReturn`, `NewType`, and types for async code. It also discusses how to give functions more precise types using overloads. All of these are only situationally useful, so feel free to skip this section and come back when you have a need for some of them.

Here's a quick summary of what's covered here:

- `NoReturn` lets you tell mypy that a function never returns normally.
- `NewType` lets you define a variant of a type that is treated as a separate type by mypy but is identical to the original type at runtime. For example, you can have `UserId` as a variant of `int` that is just an `int` at runtime.
- `@overload` lets you define a function that can accept multiple distinct signatures. This is useful if you need to encode a relationship between the arguments and the return type that would be difficult to express normally.
- Async types let you type check programs using `async` and `await`.

1.15.1 The `NoReturn` type

Mypy provides support for functions that never return. For example, a function that unconditionally raises an exception:

```
from typing import NoReturn

def stop() -> NoReturn:
    raise Exception('no way')
```

Mypy will ensure that functions annotated as returning `NoReturn` truly never return, either implicitly or explicitly. Mypy will also recognize that the code after calls to such functions is unreachable and will behave accordingly:

```
def f(x: int) -> int:
    if x == 0:
        return x
    stop()
    return 'whatever works' # No error in an unreachable block
```

In earlier Python versions you need to install `typing_extensions` using `pip` to use `NoReturn` in your code. Python 3 command line:

```
python3 -m pip install --upgrade typing_extensions
```

1.15.2 `NewTypes`

There are situations where you may want to avoid programming errors by creating simple derived classes that are only used to distinguish certain values from base class instances. Example:

```
class UserId(int):
    pass
```

(continues on next page)

(continued from previous page)

```
def get_by_user_id(user_id: UserId):
    ...
```

However, this approach introduces some runtime overhead. To avoid this, the typing module provides a helper object `NewType` that creates simple unique types with almost zero runtime overhead. Mypy will treat the statement `Derived = NewType('Derived', Base)` as being roughly equivalent to the following definition:

```
class Derived(Base):
    def __init__(self, _x: Base) -> None:
        ...
```

However, at runtime, `NewType('Derived', Base)` will return a dummy callable that simply returns its argument:

```
def Derived(_x):
    return _x
```

Mypy will require explicit casts from `int` where `UserId` is expected, while implicitly casting from `UserId` where `int` is expected. Examples:

```
from typing import NewType

UserId = NewType('UserId', int)

def name_by_id(user_id: UserId) -> str:
    ...

UserId('user')           # Fails type check

name_by_id(42)           # Fails type check
name_by_id(UserId(42))   # OK

num: int = UserId(5) + 1
```

`NewType` accepts exactly two arguments. The first argument must be a string literal containing the name of the new type and must equal the name of the variable to which the new type is assigned. The second argument must be a properly subclassable class, i.e., not a type construct like a *union type*, etc.

The callable returned by `NewType` accepts only one argument; this is equivalent to supporting only one constructor accepting an instance of the base class (see above). Example:

```
from typing import NewType

class PacketId:
    def __init__(self, major: int, minor: int) -> None:
        self._major = major
        self._minor = minor

TcpPacketId = NewType('TcpPacketId', PacketId)

packet = PacketId(100, 100)
tcp_packet = TcpPacketId(packet) # OK

tcp_packet = TcpPacketId(127, 0) # Fails in type checker and at runtime
```

You cannot use `isinstance()` or `issubclass()` on the object returned by `NewType`, nor can you subclass an object returned by `NewType`.

Note

Unlike type aliases, `NewType` will create an entirely new and unique type when used. The intended purpose of `NewType` is to help you detect cases where you accidentally mixed together the old base type and the new derived type.

For example, the following will successfully typecheck when using type aliases:

```
UserId = int

def name_by_id(user_id: UserId) -> str:
    ...

name_by_id(3) # ints and UserId are synonymous
```

But a similar example using `NewType` will not typecheck:

```
from typing import NewType

UserId = NewType('UserId', int)

def name_by_id(user_id: UserId) -> str:
    ...

name_by_id(3) # int is not the same as UserId
```

1.15.3 Function overloading

Sometimes the arguments and types in a function depend on each other in ways that can't be captured with a *union types*. For example, suppose we want to write a function that can accept x-y coordinates. If we pass in just a single x-y coordinate, we return a `ClickEvent` object. However, if we pass in two x-y coordinates, we return a `DragEvent` object.

Our first attempt at writing this function might look like this:

```
def mouse_event(x1: int,
                y1: int,
                x2: int | None = None,
                y2: int | None = None) -> ClickEvent | DragEvent:
    if x2 is None and y2 is None:
        return ClickEvent(x1, y1)
    elif x2 is not None and y2 is not None:
        return DragEvent(x1, y1, x2, y2)
    else:
        raise TypeError("Bad arguments")
```

While this function signature works, it's too loose: it implies `mouse_event` could return either object regardless of the number of arguments we pass in. It also does not prohibit a caller from passing in the wrong number of ints: mypy would treat calls like `mouse_event(1, 2, 20)` as being valid, for example.

We can do better by using **overloading** which lets us give the same function multiple type annotations (signatures) to more accurately describe the function's behavior:

```

from typing import overload

# Overload *variants* for 'mouse_event'.
# These variants give extra information to the type checker.
# They are ignored at runtime.

@overload
def mouse_event(x1: int, y1: int) -> ClickEvent: ...
@overload
def mouse_event(x1: int, y1: int, x2: int, y2: int) -> DragEvent: ...

# The actual *implementation* of 'mouse_event'.
# The implementation contains the actual runtime logic.
#
# It may or may not have type hints. If it does, mypy
# will check the body of the implementation against the
# type hints.
#
# Mypy will also check and make sure the signature is
# consistent with the provided variants.

def mouse_event(x1: int,
                y1: int,
                x2: int | None = None,
                y2: int | None = None) -> ClickEvent | DragEvent:
    if x2 is None and y2 is None:
        return ClickEvent(x1, y1)
    elif x2 is not None and y2 is not None:
        return DragEvent(x1, y1, x2, y2)
    else:
        raise TypeError("Bad arguments")

```

This allows mypy to understand calls to `mouse_event` much more precisely. For example, mypy will understand that `mouse_event(5, 25)` will always have a return type of `ClickEvent` and will report errors for calls like `mouse_event(5, 25, 2)`.

As another example, suppose we want to write a custom container class that implements the `__getitem__` method (`[]` bracket indexing). If this method receives an integer we return a single item. If it receives a slice, we return a `Sequence` of items.

We can precisely encode this relationship between the argument and the return type by using overloads like so (Python 3.12 syntax):

```

from collections.abc import Sequence
from typing import overload

class MyList[T](Sequence[T]):
    @overload
    def __getitem__(self, index: int) -> T: ...

    @overload
    def __getitem__(self, index: slice) -> Sequence[T]: ...

    def __getitem__(self, index: int | slice) -> T | Sequence[T]:

```

(continues on next page)

(continued from previous page)

```

if isinstance(index, int):
    # Return a T here
elif isinstance(index, slice):
    # Return a sequence of Ts here
else:
    raise TypeError(...)

```

Here is the same example using the legacy syntax (Python 3.11 and earlier):

```

from collections.abc import Sequence
from typing import TypeVar, overload

T = TypeVar('T')

class MyList(Sequence[T]):
    @overload
    def __getitem__(self, index: int) -> T: ...

    @overload
    def __getitem__(self, index: slice) -> Sequence[T]: ...

    def __getitem__(self, index: int | slice) -> T | Sequence[T]:
        if isinstance(index, int):
            # Return a T here
        elif isinstance(index, slice):
            # Return a sequence of Ts here
        else:
            raise TypeError(...)

```

Note

If you just need to constrain a type variable to certain types or subtypes, you can use a *value-constrained type variable*.

The default values of a function’s arguments don’t affect its signature – only the absence or presence of a default value does. So in order to reduce redundancy, it’s possible to replace default values in overload definitions with ... as a placeholder:

```

from typing import overload

class M: ...

@overload
def get_model(model_or_pk: M, flag: bool = ...) -> M: ...
@overload
def get_model(model_or_pk: int, flag: bool = ...) -> M | None: ...

def get_model(model_or_pk: int | M, flag: bool = True) -> M | None:
    ...

```


Runtime behavior

An overloaded function must consist of two or more overload *variants* followed by an *implementation*. The variants and the implementations must be adjacent in the code: think of them as one indivisible unit.

The variant bodies must all be empty; only the implementation is allowed to contain code. This is because at runtime, the variants are completely ignored: they're overridden by the final implementation function.

This means that an overloaded function is still an ordinary Python function! There is no automatic dispatch handling and you must manually handle the different types in the implementation (e.g. by using `if` statements and `isinstance` checks).

If you are adding an overload within a stub file, the implementation function should be omitted: stubs do not contain runtime logic.

Note

While we can leave the variant body empty using the `pass` keyword, the more common convention is to instead use the ellipsis (`...`) literal.

Type checking calls to overloads

When you call an overloaded function, mypy will infer the correct return type by picking the best matching variant, after taking into consideration both the argument types and arity. However, a call is never type checked against the implementation. This is why mypy will report calls like `mouse_event(5, 25, 3)` as being invalid even though it matches the implementation signature.

If there are multiple equally good matching variants, mypy will select the variant that was defined first. For example, consider the following program:

```
# For Python 3.8 and below you must use `typing.List` instead of `list`. e.g.
# from typing import List
from typing import overload

@overload
def summarize(data: list[int]) -> float: ...

@overload
def summarize(data: list[str]) -> str: ...

def summarize(data):
    if not data:
        return 0.0
    elif isinstance(data[0], int):
        # Do int specific code
    else:
        # Do str-specific code

# What is the type of 'output'? float or str?
output = summarize([])
```

The `summarize([])` call matches both variants: an empty list could be either a `list[int]` or a `list[str]`. In this case, mypy will break the tie by picking the first matching variant: `output` will have an inferred type of `float`. The implementer is responsible for making sure `summarize` breaks ties in the same way at runtime.

However, there are two exceptions to the “pick the first match” rule. First, if multiple variants match due to an argument being of type `Any`, mypy will make the inferred type also be `Any`:

```
dynamic_var: Any = some_dynamic_function()

# output2 is of type 'Any'
output2 = summarize(dynamic_var)
```

Second, if multiple variants match due to one or more of the arguments being a union, mypy will make the inferred type be the union of the matching variant returns:

```
some_list: list[int] | list[str]

# output3 is of type 'float | str'
output3 = summarize(some_list)
```

Note

Due to the “pick the first match” rule, changing the order of your overload variants can change how mypy type checks your program.

To minimize potential issues, we recommend that you:

1. Make sure your overload variants are listed in the same order as the runtime checks (e.g. `isinstance` checks) in your implementation.
2. Order your variants and runtime checks from most to least specific. (See the following section for an example).

Type checking the variants

Mypy will perform several checks on your overload variant definitions to ensure they behave as expected. First, mypy will check and make sure that no overload variant is shadowing a subsequent one. For example, consider the following function which adds together two `Expression` objects, and contains a special-case to handle receiving two `Literal` types:

```
from typing import overload

class Expression:
    # ...snip...

class Literal(Expression):
    # ...snip...

# Warning -- the first overload variant shadows the second!

@overload
def add(left: Expression, right: Expression) -> Expression: ...

@overload
def add(left: Literal, right: Literal) -> Literal: ...

def add(left: Expression, right: Expression) -> Expression:
    # ...snip...
```

While this code snippet is technically type-safe, it does contain an anti-pattern: the second variant will never be selected!

If we try calling `add(Literal(3), Literal(4))`, mypy will always pick the first variant and evaluate the function call to be of type `Expression`, not `Literal`. This is because `Literal` is a subtype of `Expression`, which means the “pick the first match” rule will always halt after considering the first overload.

Because having an overload variant that can never be matched is almost certainly a mistake, mypy will report an error. To fix the error, we can either 1) delete the second overload or 2) swap the order of the overloads:

```
# Everything is ok now -- the variants are correctly ordered
# from most to least specific.

@overload
def add(left: Literal, right: Literal) -> Literal: ...

@overload
def add(left: Expression, right: Expression) -> Expression: ...

def add(left: Expression, right: Expression) -> Expression:
    # ...snip...
```

Mypy will also type check the different variants and flag any overloads that have inherently unsafely overlapping variants. For example, consider the following unsafe overload definition:

```
from typing import overload

@overload
def unsafe_func(x: int) -> int: ...

@overload
def unsafe_func(x: object) -> str: ...

def unsafe_func(x: object) -> int | str:
    if isinstance(x, int):
        return 42
    else:
        return "some string"
```

On the surface, this function definition appears to be fine. However, it will result in a discrepancy between the inferred type and the actual runtime type when we try using it like so:

```
some_obj: object = 42
unsafe_func(some_obj) + " danger danger" # Type checks, yet crashes at runtime!
```

Since `some_obj` is of type `object`, mypy will decide that `unsafe_func` must return something of type `str` and concludes the above will type check. But in reality, `unsafe_func` will return an `int`, causing the code to crash at runtime!

To prevent these kinds of issues, mypy will detect and prohibit inherently unsafely overlapping overloads on a best-effort basis. Two variants are considered unsafely overlapping when both of the following are true:

1. All of the arguments of the first variant are potentially compatible with the second.
2. The return type of the first variant is *not* compatible with (e.g. is not a subtype of) the second.

So in this example, the `int` argument in the first variant is a subtype of the `object` argument in the second, yet the `int` return type is not a subtype of `str`. Both conditions are true, so mypy will correctly flag `unsafe_func` as being unsafe.

Note that in cases where you ignore the overlapping overload error, mypy will usually still infer the types you expect at callsites.

However, mypy will not detect *all* unsafe uses of overloads. For example, suppose we modify the above snippet so it calls `summarize` instead of `unsafe_func`:

```
some_list: list[str] = []
summarize(some_list) + "danger danger" # Type safe, yet crashes at runtime!
```

We run into a similar issue here. This program type checks if we look just at the annotations on the overloads. But since `summarize(...)` is designed to be biased towards returning a float when it receives an empty list, this program will actually crash during runtime.

The reason mypy does not flag definitions like `summarize` as being potentially unsafe is because if it did, it would be extremely difficult to write a safe overload. For example, suppose we define an overload with two variants that accept types `A` and `B` respectively. Even if those two types were completely unrelated, the user could still potentially trigger a runtime error similar to the ones above by passing in a value of some third type `C` that inherits from both `A` and `B`.

Thankfully, these types of situations are relatively rare. What this does mean, however, is that you should exercise caution when designing or using an overloaded function that can potentially receive values that are an instance of two seemingly unrelated types.

Type checking the implementation

The body of an implementation is type-checked against the type hints provided on the implementation. For example, in the `MyList` example up above, the code in the body is checked with argument list `index: int | slice` and a return type of `T | Sequence[T]`. If there are no annotations on the implementation, then the body is not type checked. If you want to force mypy to check the body anyways, use the `--check-untyped-defs` flag ([more details here](#)).

The variants must also be compatible with the implementation type hints. In the `MyList` example, mypy will check that the parameter type `int` and the return type `T` are compatible with `int | slice` and `T | Sequence` for the first variant. For the second variant it verifies the parameter type `slice` and the return type `Sequence[T]` are compatible with `int | slice` and `T | Sequence`.

Note

The overload semantics documented above are new as of mypy 0.620.

Previously, mypy used to perform type erasure on all overload variants. For example, the `summarize` example from the previous section used to be illegal because `list[str]` and `list[int]` both erased to just `list[Any]`. This restriction was removed in mypy 0.620.

Mypy also previously used to select the best matching variant using a different algorithm. If this algorithm failed to find a match, it would default to returning `Any`. The new algorithm uses the “pick the first match” rule and will fall back to returning `Any` only if the input arguments also contain `Any`.

Conditional overloads

Sometimes it is useful to define overloads conditionally. Common use cases include types that are unavailable at runtime or that only exist in a certain Python version. All existing overload rules still apply. For example, there must be at least two overloads.